PRGUE Schemes: Efficient Updatable Encryption With Robust Security From Symmetric Primitives

Elena Andreeva
TU Wien
Vienna, Austria
elena.andreeva@tuwien.ac.at

Andreas Weninger
TU Wien
Vienna, Austria
andreas.weninger@tuwien.ac.at

Abstract

Securing sensitive data for long-term storage in the cloud is a challenging problem. Updatable encryption (UE) enables changing the encryption key of encrypted data *in the cloud* while the plaintext and all versions of the key remain secret from the cloud storage provider, making it an efficient alternative for companies that seek to outsource their data storage.

The most secure UE schemes to date follow robust security models, such as the one by Boyd et al. from CRYPTO 2020, and rely exclusively on asymmetric cryptography, thus incurring a substantial performance cost. In contrast, the Nested UE construction of Boneh et al. from ASIACRYPT 2020 achieves much better efficiency with symmetric cryptography, but it provides weaker security guarantees. Boyd et al. further suggest that attaining robust UE security inherently requires the use of asymmetric cryptography.

In this work, we show for the first time that symmetric UE schemes are not inherently limited in their security and can achieve guarantees on par with, and even beyond, Boyd's UE model. To this end, we extend Boyd's framework to encompass the class of ciphertext-dependent UE schemes and introduce indistinguishability-from-random (IND$) as a stronger refinement of indistinguishability. While our IND$ notion primarily streamlines the proofs of advanced security properties within the model, it yields practical privacy advantages: ciphertexts do not exhibit a recognizable structure that could otherwise distinguish them from arbitrary data.

We then introduce two robustly secure symmetric UE constructions, tailored to different target security levels. Our schemes are built on a *novel design paradigm* that combines symmetric authenticated encryption with ciphertext re-randomization, leveraging for the first time the use of pseudorandom number generators in a *one-time-pad* style. This approach enables both robust security and high efficiency, including in AES-based implementations.

Our first scheme, PUE-List, delivers encryption up to 600x faster than prior asymmetric schemes of similar robustness, while matching Boneh et al.'s efficiency and achieving the stronger security level of Boyd et al. Our second scheme, PUE-One, further boosts performance with constant-time decryption 24x faster than all previously known UE schemes, overcoming the main bottleneck in Boneh's design, while trading off some security, yet still significantly surpassing the guarantees of Boneh's Nested scheme.

Keywords

updatable encryption, symmetric cryptography, pseudo-random generator, secure cloud storage

This is the full version of our paper that appears at AsiaCCS 2026.

1 Introduction

Organizations across sectors must retain sensitive data for extended periods due to regulatory and operational requirements, while also ensuring periodic cryptographic key rotation to maintain security over time. Beyond government, finance, and healthcare, this includes legal, educational, research, energy, and enterprise domains, among others, where intellectual property and compliance records must be preserved. On-premises, key rotation incurs only local computational cost through decryption and re-encryption of data. In cloud environments such as Google Cloud or AWS, the process requires downloading and re-uploading entire datasets, leading to significant bandwidth overhead and increased expenses for organizations.

Key Rotation: Before data is uploaded to the cloud, it is typically encrypted to mitigate risks from potentially untrusted providers and server compromise. However, static encryption alone is insufficient, as the security guarantees collapse if the encryption keys are exposed. To reduce this risk, cryptographic keys are periodically rotated. *Key rotation* is mandated in official guidelines, such as for example by NIST [7], CMS [17] or PCI DSS [13]. In order to satisfy NIST recommendations, a symmetric encryption key should be used for a maximum time of 2 years or less, depending on the volumes of data that are encrypted ([7], page 40). The U.S. Federal Reserve mandates the retention of files such as actuarial valuation reports for up to 30 years [16]. Consequently, a ciphertext may undergo periodic re-encryption, for example on an annual basis, throughout the entire retention period. Real-world threats of key leakage arising from insecure key storage or weak key derivation underscore the necessity of robust cryptographic key management and secure cloud storage mechanisms beyond the application of simple encryption.

The naïve way to realize key rotation is to download the data, decrypt it, and then re-encrypt it with a new key before uploading it again. Yet, this approach incurs high bandwidth cost, which is avoidable by performing all operations locally on the server, without revealing the encrypted data to the cloud storage provider in the process. This observation motivates the more practical approach of *updatable encryption* (UE).

Updatable Encryption (UE): An UE scheme (see Figure 1) fulfils the usual symmetric encryption Enc and decryption Dec functionalities, and additionally introduces procedures for *key rotation* Gen and token generation TG. Here, we describe the high level of interaction in the so-called *ciphertext-dependent* UE. First, the **client** C generates locally a key k and encrypts its data m as the pair of header and ciphertext $(\hat{c}, c) = \mathsf{Enc}(k, m)$. $(\hat{c}, c)$ is then sent to the **server** $\mathcal{S}$ and disposed of by C. To rotate the key k, $\mathcal{S}$ sends $\hat{c}$ to C, who in turn applies the algorithms Gen to generate a new key k'

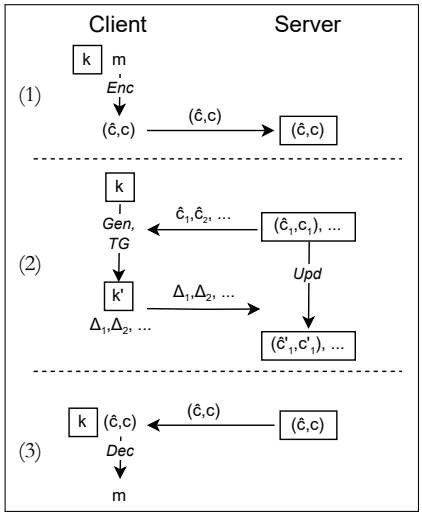

Figure 1: Usage of (ciphertext-dependent) UE schemes. Boxed values need to be stored.

and TG to generate a small ciphertext update token Δ. The token Δ is specific to the ciphertext $(\hat{c}, c)$ (hence the name *ciphertext-dependent* UE scheme). C then deletes the old key k. $\mathcal{S}$ receives Δ and updates the header and ciphertext as $(\hat{c}', c') = \mathsf{Upd}(\Delta, (\hat{c}, c))$. The time period during which the client has any single key is considered one epoch. Gen, TG and Upd are repeated as many times as key rotations performed. In our example the new k' is used to decrypt c' as $m = \mathsf{Dec}(k', c')$.

UE Performance: The performance of UE schemes is measured in terms of *computation, storage* and *bandwidth* costs. The algorithms Enc and Dec at the client's end, and the ciphertext update Upd at the server's end are computationally heavier, while Gen and TG are significantly lighter. According to NIST recommendations [7], the update computations may occur only once per year. On the other hand, storage and bandwidth are used continuously, and are directly affected by ciphertext overhead. As such, an overhead of up to 36% in existing schemes [9] is a significant cost factor.

UE Design Approaches: Nearly all proposed UE schemes to date rely on techniques from asymmetric cryptography, such as Learning With Errors (LWE) [24], the Diffie-Hellman assumption [10], and key-homomorphic primitives [9, 12, 15, 22]. Only a limited number of symmetric UE constructions exist, based on symmetric authenticated encryption schemes [8, 15, 22]. While symmetric approaches offer substantial performance advantages, they fail to provide robust security guarantees, in the way asymmetric UE schemes do. A promising and novel direction would be to rethink the mechanisms for ciphertext updates that are used in symmetric UE schemes, and to aim to enable fast and robustly secure updates in a one-time pad-style manner, for example, by leveraging a high-speed pseudorandom number generator (PRNG).

UE Security: The key rotation mandated by NIST [7] is traditionally carried out by decrypting and re-encrypting the ciphertext under a new key, resulting in a fresh ciphertext. For UE schemes to serve as an equally secure alternative, the updated ciphertext must also be fully "refreshed". This notion has been formalized and broken down into several properties in UE security models. Fundamentally, a UE scheme has to remain secure even when the *key is leaked* at some point in time. A basic security requirement for *any* UE scheme is *encryption indistinguishability* (IND-ENC) under CPA (chosen plaintext attack), which extends standard confidentiality of authenticated encryption schemes to UE schemes. UE security models aim to also capture the following security properties:

(A) **Update indistinguishability (IND-UPD).** Intuitively, IND-UPD captures the idea that the update procedure re-randomizes the ciphertexts and creates *unlinkability* between a ciphertext and its updated version. For example, if an attacker obtains ciphertexts at two different points in time, they should not be able to determine whether the underlying plaintexts are identical. Consider a company storing data in the cloud: if an employee with knowledge of all previously stored ciphertexts leaves the organization, they should not be able to infer any relationship between the current ciphertexts and those they previously knew.

(A+) **Ciphertext age hiding.** This property is a refinement of (A) and captures the idea that fresh ciphertexts should not be distinguishable from already updated ciphertexts. In other words, the ciphertext age (i.e. the number of updates) should not be discernable, a threat that is relevant, for example, when an attacker knows that some newspaper interacted with their informant recently and stored the contact data in the cloud. By checking the ciphertext age the attacker should not be able to deduce whether the newspaper has been in contact with this informant before. Another example is revealing the age of a person in the citizen database or how long a user has been registered in a dating app.

(B) **Adaptive corruptions.** The adversary can decide at any point to corrupt the current key. In models with non-adaptive corruptions, the adversary must declare whether the next key is corrupted before going into an epoch.

(C) **CCA security.** The adversary can send ciphertexts of their choice to the client for decryption. This may occur in practice if the attacker injects its own ciphertext into the server storage or into the transmitted to the server ciphertexts.

We say a UE scheme has robust security if it achieves all properties (A), (A+), (B) and (C).

UE Security Models and Schemes. We demonstrate a security comparison for the UE distinct schemes in Table 2. A simple solution to UE is the so-called *hybrid encryption scheme* [8, 15] (HES) that is used in a similar form by Amazon AWS [1] and Google [19]. In HES updates happen only over $\hat{c}$ which is a significant computational simplification and cost optimization, but it comes with security disadvantages. In fact, c becomes easily *linkable* across updates since it never changes (no (A)) and ciphertexts are not fully re-randomized. HES, hence, fails to meet the requirements by NIST for periodic rotation of data encryption keys since the key for c is never changed despite c potentially remaining on the server for extended periods of up to 30 years. Additionally, if an old $(\hat{c}, c)$ and k become known, an attacker that is given the updated $(\hat{c}', c')$ can

break the integrity in HES by forging a new valid c'' for the current, unknown key (fails property (C)).

Although the NIST requirements do not explicitly define the precise security guarantees expected after rekeying, they implicitly assume fully refreshed ciphertexts that, in particular, prevent forgeries. Consequently, existing UE constructions, such as HES, fall short of meeting such security. In practice, users are often compelled to adopt the costly procedure of downloading all data, decrypting and re-encrypting it under a new key, and subsequently re-uploading it.

The term updatable encryption (UE) was coined in the work of Boneh et al. [9] (BLMR13) and the first UE formalization was done by Everspaugh et al. [15]. Their security model captures (A) via a static list of honest and corrupt keys which prohibits adaptive corruptions. Everspaugh et al. propose KSS as a variant of HES that overcomes the integrity problem. However, KSS, similar to HES, has a ciphertext component that is never re-randomized and it fails to provide ciphertext unlinkability (no (A)). For this reason, Everspaugh et al. additionally propose ReCrypt, which is based on key-homomorphic primitives, and achieves (A).

Boneh et al. [8] (BEKS20) strengthen the model of Everspaugh et al. [15], by introducing *ciphertext age hiding* (A+). BEKS20 were the first to also propose a symmetric scheme, the Nested UE scheme, that provides (A,A+) albeit no (B,C). The security of the Nested scheme is limited by the fact that tokens cannot be adversarially corrupted, which we will refer to as *relaxed security setting*. Such setting is practically meaningful when tokens are sent over secure channels (TLS). BEKS20 additionally introduce the asymmetric UE scheme KH-PRF. The Nested scheme uses AES and significantly outperforms other solutions, with encryptions being 13-47x faster than KH-PRF [8]. Decryption slows down linearly with the number of ciphertext updates and eventually become slower than asymmetric UE schemes after 50 or more updates. The Nested scheme is hence suitable for applications that follow the NIST recommendations (around 30 updates in total), but less so when a much larger number of updates is needed or the relaxed security setting is insufficient.

It has since remained an open question whether there exist purely symmetric UE schemes with robust security (close to that by asymmetric UE schemes), encryption performance comparable to the Nested scheme and no inherent blowup in decryption time.

In a different line of work, Lehmann et al. [22] propose a distinct UE security model, which is to first to allow adaptive corruptions (B). Further, Lehmann et al. [22] consider ciphertext-independent UE, which requires a single token to update *any* of the current ciphertexts in contrast to prior ciphertext-dependent UE with one token per ciphertext. While ciphertext-independent token generation would seem more efficient, the overall performance of concrete UE schemes depends much more on other factors, such as the choice of the underlying primitives and techniques to achieve UE security. While achieving (A) and (B) security, the UE schemes by Lehmann et al. do not achieve CCA (C) security. In particular their 2ENC scheme, which is purely symmetrical, has easily malleable ciphertexts (no CCA security (C)) and, when tokens are leaked, neither achieves unlinkability (A) nor the fundamental IND-ENC property.

This gap was addressed by Klooß et al. [21] who provide a model for ciphertext-independent UE with CCA security. However, the security of their UE schemes rely on restrictions on CCA adversaries and asymmetric instantiations with performance far worse than the Nested design.

In 2020 Boyd et al. [10] strengthened the model by Lehmann et al. [22] with CCA security, and proved the generic composition CPA + INT-CTXT $\Rightarrow$ CCA using their newly introduced integrity (INT-CTXT) notion. Additionally, they introduce (A+) similarly to BEKS20. Boyd's model is robust as it allows an adversary to access all keys, ciphertexts and update tokens, and is only limited by some trivial win conditions. Overall, Boyd's model captures properties (A), (A+),(B) and (C). As a downside, their formalization of the (A+) property, IND-UE-CPA, cannot be broken into smaller components, which causes large monolithic proofs. Also, their considered adversaries can corrupt tokens *from previous epochs*, despite these ephemeral tokens being immediately deleted after use in practice. In terms of UE schemes, Boyd et al. propose the SHINE family. SHINE scores computationally significantly worse (more than 100x slower) than the Nested scheme [9].

To date, it has remained unanswered whether an efficient, purely symmetric UE scheme can achieve robust security. Prior work suggested asymmetric techniques are necessary [10, 24], and existing impossibility results apply only to ciphertext-independent UE [2]. Yet, despite a decade of research, the case of symmetric ciphertext-dependent UE schemes remains open.

1.1 Contributions

UE security model. Our UE security model is based on the established model by Boyd et al. [10]. We adapt it to encompass the syntax of ciphertext-dependent UE schemes. Ciphertext-dependent UE schemes have received security treatment in a separate line of UE works [11, 21, 22].

We consider *chronological adversaries* that may corrupt tokens only in the epoch in which they are generated. While such adversaries can ultimately obtain the same set of tokens as in Boyd's model, they must do so in the corresponding epoch rather than retroactively, for example after observing ciphertexts from later epochs. This departs from prior work that allows retroactive token corruption [10, 11, 21, 22].

Nevertheless, to the best of our current understanding, this restriction still captures all practically relevant attacks. Tokens are ephemeral values that are transmitted once and intended for immediate use, making retrospective exposure significantly less likely than for long-term secrets such as keys or ciphertexts stored persistently on disk. Moreover, attackers that exfiltrate a predetermined set of prior tokens—whether all tokens, the most recent tokens, or a random subset—can be equivalently modeled as corrupting those tokens in earlier epochs, prior to server compromise. Retroactive corruption therefore only provides additional power when performed adaptively based on information unavailable beforehand, which does not reflect realistic attack scenarios, where attackers would compromise as many tokens as possible, limited only by the given attack vector, rather than adaptively choosing to corrupt only a subset of tokens.

Additionally, we introduce two new indistinguishability-from-random (IND$) security notions, defined as stronger variants of the standard indistinguishability (IND) notions. Intuitively, given either a message (to be encrypted) or a ciphertext (to be updated),

no adversary should be able to distinguish the corresponding real-world output from a uniformly random string. Formulating this definition is non-trivial, as in the ideal world the adversary receives only random strings, yet must still be provided with update tokens and updated ciphertexts that remain consistent with the scheme's functionality. We show that our IND$ notions strictly imply their IND counterparts. A key advantage of adopting the IND$ notions is that ciphertext age hiding is obtained *"for free"*, eliminating the need for a separate experiment for property (A+). Specifically, if ciphertexts of arbitrary age are indistinguishable from random strings, then they are necessarily indistinguishable from each other. We formally prove this implication and further show that our ciphertext age hiding property implies Boyd's formulation of (A+) (IND-UE-CCA). Moreover, in practical settings—such as local file servers maintaining cloud-based copies of UE ciphertexts—our IND$ notions provide stronger privacy guarantees, since they prevent UE ciphertexts from exhibiting structural patterns that an adversary could exploit to distinguish them from unrelated data.

With these adjustments in mind, our model, similarly to Boyd et al., covers all modern UE security features, namely (A), (A+), (B), and (C).

Novel UE schemes. We propose the first symmetric-based UE schemes that are both highly efficient and achieve robust security. Our schemes use symmetric authenticated encryption (AE) and ciphertext updates are realized by deploying a *novel* design approach by XORing the output of a pseudo-random generator (PRG) onto the ciphertext in a one-time-pad manner. Our two UE schemes PUE-One and PUE-List exhibit distinct trade-offs between performance, security, and functionality (see Table 1).

Scheme	Security Level	Encrypt	Decrypt (1 - 50 u)	Update	Overhead
SHINE [10]	CCA	≈ 15000	≈ 13000	≈ 12000	3%
PUE-List (this work)	CCA	19.63	19.07 391.56	7.78	3%
ReCrypt [15]	CPA	15431	13745	12945	3%
KH-PRF [8]	CPA	60.06	33.72	19.80	36%
Nested [8]	CPA, no token	20.43	20.35 405.18	10.27	7%
PUE-One (this work)	CCA, no token	20.04	16.94	14.53	0.3%

Table 1: Performance, security and ciphertext expansion. Measurements are in cycles per byte, lower is better. We grouped the schemes by security level, with the high security schemes in the first box. "no token " refers to restricting the adversary from corrupting tokens. Multiple decrypt values refer to the rate after 1 update and after 50 updates. Ciphertext expansion is considered for 32KB messages.

Scheme	Type	(A)	(B)	(C)	Corrupt Token	IND$
SHINE [10]	ciph-indep	✓	✓	✓	✓	·
PUE-List (this work)	ciph-dep	✓	✓	✓	✓	✓
ReCrypt [15]	ciph-dep	✓	·	·	✓	·
KH-PRF [8]	ciph-dep	✓	·	·	✓	·
Nested [8]	ciph-dep	✓	·	·	·	·
PUE-One (this work)	ciph-dep	✓	✓	✓	·	✓

Table 2: Security Comparison of UE schemes. Properties (A) to (C) are discussed in the introduction. Schemes are proven secure in models with the indicated adversarial capabilities ✓. No checkmark (·) indicates a missing security proof with this property, we make no claim about a vulnerability. Type is ciphertext-dependent or ciphertext-independent.

1. PUE-One[1] focuses on *encryption and decryption speed* as these (a) are executed on each client; (b) can be called multiple times per day, and (c) directly impact the response time of the system for an end user. Token generation and update speed are of a secondary importance as these are executed only once per year (NIST example) and can be run during downtime. PUE-One also minimizes ciphertext overhead towards lowering storage cost and network bandwidth consumption.

PUE-One surpasses the security of the Nested scheme [8], by additionally achieving B, C and IND$ security. Similar to the Nested scheme, PUE-One assumes no token corruption, but otherwise still maintains basic encryption indistinguishability (IND-ENC). In comparison to the Nested scheme, PUE-One is marginally faster in encryption and 41% faster in token generation. The main advantage of PUE-One is that it achieves *constant decryption time*, resulting in 24x faster decryptions than the Nested scheme after 50 updates. Contrary to the Nested scheme, PUE-One allows for an unlimited number of updates. On the downside, PUE-One is 46% slower during updates. PUE-One provides a practical solution that avoids the security shortcomings of currently deployed HES schemes (AWS [1], Google [18]), namely the lack of properties (A) and (C). Moreover, it achieves competitive performance, being at most 1.6x slower than AES, in contrast to prior robustly secure UE schemes whose encryption and decryption incur a slowdown of at least 5x.

2. PUE-List[2] achieves *full security* in our model and thus is the first purely symmetric UE scheme to fulfill the robust security of modern UE models such as Boyd et al. (in our adaptation). Additionally,

[1]Named for its use of a single PRG seed in the ciphertext blinding process.
[2]Named after the used list of seeds per ciphertext.

PUE-List achieves our new IND$ security. Unlike PUE-One, this holds even when the adversary can corrupt tokens.

Compared to its robustly secure competitors, the SHINE schemes, PUE-List vastly outperforms them with 1500x faster updates, 600x faster encryption and 600x-30x faster decryption (for 1 to 50 updates). PUE-List surpasses the Nested scheme by achieving robust security, but has similar performance and also requires setting the maximum number of updates per ciphertext. When storing a ciphertext for 30 years with annual key rotation, 30 updates suffice. Up to this point, PUE-List guarantees full security. Afterwards, the ciphertext cannot be updated further. To keep it in the system, the ciphertext needs to be decrypted and reentered as a fresh message.

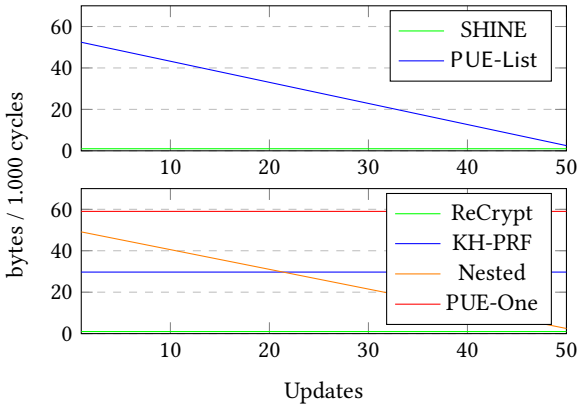

Figure 2: Decryption throughput of high security schemes (at the top) and relaxed security schemes (at the bottom). Higher is better.

We illustrate a security comparison between our novel and existing UE schemes in Table 2. The table contains only schemes that achieve at least security property (A). Several less secure schemes (HES, KSS and 2ENC) do not have an implementation available. For HES and KSS, which have constant-time updates, we estimate encryption and decryption to be 1.6x faster than PUE-One. For 2ENC we estimate 1.1x slower encryptions, 1.3x slower decryptions and 1.6x slower updates than PUE-One.

Proof techniques. Our model allows the adversary to adaptively corrupt keys after observing ciphertexts, making it a challenging task to reduce the security of our schemes to the security of the underlying AE. The naïve approach of guessing when the key is corrupted leads to an advantage term $2^{e_{max}}$ (e_{max} is the number of ciphertext updates). To address this issue, we introduce DIC, a security notion in the spirit of AE security, with an additional key corruption oracle. We prove DIC to be equivalent to AE security, which in turns allows us to reduce the advantage term to e_{max} instead of $2^{e_{max}}$. Overall, we are able to reduce the security of our schemes to standard assumptions on symmetric primitives, such as AE privacy and authenticity, a collision resistance of hash functions and a secure pseudo-random generator.

1.2 Related Works

Chen and Jiang [11] focus on ciphertext-dependent UE models and consider a security model for adaptive security, deterministic updates and CCA security. Our model also encompasses these features as inherited from the model by Boyd et al. A line works [20, 23, 24] proposes security models similar to Boyd et al. (capturing properties (A),(A+),(B),(C)) that go on (orthogonally to our work) to model further additional features, for example unidirectional vs bi-directional updates. Chen et al. [12] further consider malicious ciphertexts as input to the update function, but their model does not allowing adaptive corruptions (no (B)). The security of schemes in [11, 12, 20, 23, 24] relies on e.g. the LWE assumption and is thus hard to compare to our schemes which rely on AE/hash security. Performance-wise [11, 12, 20, 23, 24] provide no benchmarks, but we estimate performances close to ReCrypt.

2 Preliminaries

Notation. Let $a \leftarrow b$ denote assigning b to the variable a, if b is a function then $\leftarrow$ does not specify whether b is deterministic or randomized. We write $a \leftarrow\!\!\$\ S$ to denote uniformly randomly sampling an element from the set S and assigning it to variable a. $|\cdot|$ denotes the size of a set or the length of a string. $A \cup B$ denotes the union of sets A and B. $\wedge, \vee$ are the logical and/or operators. $\oplus$ denotes bitwise exclusive or (XOR). $\perp$ is a special symbol, often returned as error symbol. λ is the security parameter of the system, which is assumed to be known by all algorithms. ppt is short for probabilistic polynomial time.

For an integer x, $\langle x \rangle$ denotes encoding x as a bitstring of predefined constant length. This constant is not explicitly specified, but is assumed to be sufficiently long for all such integers that could occur in the scheme in question. For a set S of size at most ℓ, $\langle S \rangle_\ell$ means encoding the set as a fixed length bitstring, which encodes $|S|$ and the individual elements as fixed length bitstrings. For a bitstring X, $X[i, \ell]$ denotes taking the substring made from the ℓ bits from indices i to $i + \ell - 1$. When we test whether a tuple is part of a set, we can use $\cdot$ as a wildcard for one or more of the slots, e.g. the expression $(a, \cdot, \cdot) \in S$ is equivalent to $\exists b, c : (a, b, c) \in S$.

Advantage. We will write advantage terms as e.g. $\mathsf{Adv}^{\mathsf{dic}}_{\mathsf{AE},\mathcal{A}}(\lambda)$ for the advantage of some adversary $\mathcal{A}$ against scheme AE in experiment DIC with security parameter λ. When omitting the adversary e.g. $\mathsf{Adv}^{\mathsf{dic}}_{\mathsf{AE}}(\lambda)$ this denotes the maximum advantage of any probabilistic polynomial time (ppt) adversary.

Encryption. An AE scheme is the tuple of algorithms AE = (AE.Gen, AE.Enc, AE.Dec), where AE.Gen and AE.Enc are probabilistic. AE.Gen takes the security parameter 1^λ and outputs key k. AE.Enc takes key k, arbitrary length message m and outputs ciphertext c. The correctness condition of AE requires $\Pr[\mathsf{AE.Dec}_k(\mathsf{AE.Enc}_k(M)) = M : k \leftarrow \mathsf{AE.Gen}(1^\lambda)] = 1$. Let oracles $\mathcal{E}_k(\cdot) = \mathsf{AE.Enc}_k(\cdot), \mathcal{D}_k(\cdot) = \mathsf{AE.Dec}_k(\cdot)$. We define AE privacy against a ppt adversary $\mathcal{A}$ as $\mathsf{Adv}^{\mathsf{priv}}_{\mathsf{AE},\mathcal{A}}(\lambda) = \left|\Pr[k \leftarrow \mathsf{AE.Gen}(1^\lambda) : \mathcal{A}^{\mathcal{E}_k(\cdot)} = 1] - \Pr[\mathcal{A}^{\$(\cdot)} = 1]\right|$ where $\$(M)$ returns a random string of length $|\mathcal{E}_k(M)|$. Further we define AE authenticity against a ppt adversary $\mathcal{A}$ as $\mathsf{Adv}^{\mathsf{auth}}_{\mathsf{AE},\mathcal{A}}(\lambda) = \left|\Pr[k \leftarrow \mathsf{AE.Gen}(1^\lambda) : \mathcal{A}^{\mathcal{E}_k(\cdot),\mathcal{D}_k(\cdot)} \text{ forg.}]\right|$ where we say that the

adversary forges (forg.) if they call $\mathcal{D}(\cdot)$ on some ciphertext C which was not previously returned by $\mathcal{E}$ and for which $\mathcal{D}(C) \neq \perp$.

A symmetric encryption scheme SE follows the same syntax as an AE scheme. $\mathsf{Adv}^{\mathrm{priv}}_{\mathsf{SE}}(\lambda)$ is defined as for AE schemes, but SE schemes are not expected to fulfill the auth notion. Our schemes rely on generic AE, and thus can be instantiated both with classical designs (AES-GCM [14]) and novel designs based on expanding primitives, such as forkciphers [5, 6, 25] and expanding PRFs (ButterKnife [4]).

PRG. A pseudorandom generator (PRG) G with seed space $\{0,1\}^z$ and output space $\{0,1\}^y$ is defined as a function $G : \{0,1\}^z \to \{0,1\}^y$. We define the PRG security against a ppt adversary $\mathcal{A}$ as

$$\mathsf{Adv}^{\mathrm{prg}}_{G,\mathcal{A}}(\lambda) = \big|\Pr[\mathsf{seed} \leftarrow\!\$ \{0,1\}^z, x \leftarrow G(\mathsf{seed}) : \mathcal{A}(x) = 1] - \Pr[x \leftarrow\!\$ \{0,1\}^y : \mathcal{A}(x) = 1]\big|.$$

Hash function. The collision security of a family of hash functions $\mathcal{H} = \{H : \{0,1\}^* \to \{0,1\}^\ell\}$ (for some $\ell \in \mathbb{N}$) is defined as

$$\mathsf{Adv}^{\mathrm{collres}}_{H} = \max_{\mathcal{A}} \Pr_{H \leftarrow\!\$ \mathcal{H}}[(x_0, x_1) \leftarrow \mathcal{A}(1^\lambda, H) \wedge H(x_0) = H(x_1)]$$

where $\mathcal{A}$ is taken from all ppt adversaries.

3 Updatable Encryption

In this section we introduce our updatable encryption syntax, which follows the standard syntax of ciphertext-dependent UE schemes [8]. See Figure 1 for a visualization.

Definition. An updatable encryption scheme (UE) consists of the following algorithms:

- $\mathsf{Gen}(1^\lambda)$: On input the security parameter 1^λ, the Gen algorithm returns a key k.
- $\mathsf{Enc}_k(m)$: Encrypts message m with a key k and returns ciphertext $C = (\hat{c}, c)$ where $\hat{c}$ is the ciphertext header and c is the ciphertext body. The ciphertext length $|C|$ must only depend on $|m|$ and the security parameter λ, and is given through some function $\ell_C(|m|)$.
- $\mathsf{Dec}_k(C)$: Decrypts C with k.
- $\mathsf{TG}(k, k', \hat{c})$: Takes the old key k, the new key k' and $\hat{c}$ to return the (ciphertext-specific) update token Δ of fixed length ℓ_Δ (which depends only on security parameter λ). For ciphertext-independent UE schemes TG returns $\perp$.
- $\mathsf{Upd}(\Delta, C)$: Takes ciphertext-specific update token Δ and C to return new ciphertext C'.

Correctness. The correctness of a UE scheme $\mathsf{UE} = (\mathsf{Gen}, \mathsf{Enc}, \mathsf{Dec}, \mathsf{TG}, \mathsf{Upd})$ requires that the encrypted messages should be decrypted correctly in the same epoch or any subsequent epoch if the ciphertexts were updated to that epoch. The code in Figure 3 should return true for any message $m \in \{0,1\}^*$ and any number of updates $u \in \{0, 1, \ldots\}$, even if additional messages m' are encrypted, decrypted or updated at any time.

Threat model. We consider strong adversaries with the ability to read any transmitted or stored data (including the key), apart from trivial wins. We ensure the confidentiality, authenticity, ciphertext age hiding and unlinkability against attackers who compromise the server and client in distinct epochs. Accessing ciphertexts on the server requires the user to login, a server compromise could be a vulnerability in this login function or the user might have used the same password for some insecure website.

$k \leftarrow \mathsf{Gen}(1^\lambda)$
$(\hat{c}, c) \leftarrow \mathsf{Enc}_k(m)$
for $i = 1 \to u$
  $k' \leftarrow \mathsf{Gen}(1^\lambda)$
  $\Delta \leftarrow \mathsf{TG}(k, k', \hat{c})$
  $k \leftarrow k'$
  $(\hat{c}, c) \leftarrow \mathsf{Upd}(\Delta, (\hat{c}, c))$
$m' \leftarrow \mathsf{Dec}_k((\hat{c}, c))$
return $m = m'$

Figure 3: Assessing correctness.

4 Security

In Section 4.2 we define the basic security model, which largely matches that of Boyd et al [10], apart from what we discuss in Section 4.3.

An informal overview over the model is in Section 4.1. In Section 4.4 we present our new features and strengthened security definitions. In Section 4.5 we show relations among the security notions. We summarize security in our model in Section 4.8.

4.1 Security Notions Overview

Our experiment contains several sets, which are described in Table 3. We give an informal overview over our notions in Tables 4 and 5.

4.2 Basic Security Model

Oracles and security notions. We outline the generic security experiment xxIND-yy-atk for indistinguishability notions in Figure 5. To instantiate it, "xx" can be either "det" or "rand", indicating deterministic or randomized ciphertext updates (Upd). Our schemes use deterministic updates. "atk" can be "CPA" or "CCA" (giving access to a decryption oracle). "yy" defines which function is attacked, either ENC (encryptions) or UPD (updates). The "Create $\tilde{C}$ with CHALL" call creates the challenge ciphertext $\tilde{C}$ based on the adversary's challenge CHALL. This call differs for each notion, as defined in Figure 6. Experiment xxIND-yy-atk can thus be instantiated as e.g. detIND-ENC-CPA. A separate security experiment (Figure 5) defines ciphertext integrity INT-CTXT.

Oracles are introduced in Figure 4. O_{Next} moves to a new epoch, generates the corresponding key and tokens and updates the challenge ciphertext. O_{Upd} updates and reveals regular ciphertexts. O_{UpdC} reveals the challenge ciphertext. O_{Try} is used in the INT-CTXT experiment to attempt to forge a ciphertext.

Experiment state variables. The experiment maintains several variables which change throughout the execution. Their behavior is controlled through the pseudocode in our figures, we give an intuition of their purpose here.

- e is the number of the current epoch (0 initially).
- $\tilde{e}$ is the epoch number in which the challenge ciphertext is created.

Set	Description
$\mathcal{L}$	Contains the regular ciphertexts in the system in the form (i, c, e), meaning c is the i-th ciphertext overall that was encrypted. This encryption either happened in epoch e or it was since updated to epoch e.
$\tilde{\mathcal{L}}$	The challenge ciphertexts and their updates, with format (c, e) saying ciphertext c was encrypted in/updated to epoch e.
$\mathcal{K}/\mathcal{T}$	The epochs e for which the key/token was leaked via oracle O_{Corr}.
$\mathbb{C}$	The epochs e for which the current version of the challenge ciphertext was leaked via $O_{\mathsf{Upd}\mathbb{C}}$.
$\mathcal{K}^*/\mathcal{T}^*/\mathbb{C}^*$	The extended sets of all epochs in which the adversary can infer the keys/tokens/ciphertexts.
$\mathcal{J}$	Contains the indices of the two ciphertexts that were used to create an IND-UPD challenge. To prevent trivial wins, the adversary cannot use oracles O_{Upd} on these ciphertexts anymore (instead $O_{\mathsf{Upd}\mathbb{C}}$) and the O_{Corr} token corruption will not give update tokens for these ciphertexts.

Table 3: Sets in our experiment.

- i is a counter that reflects the number of ciphertexts in the system and serves to assign a unique index to each ciphertext.
- $\mathsf{index}_{\tilde{C}}$ is the index of the challenge ciphertext.
- phase is 0 before the adversary provided its challenge and is set to 1 afterwards.
- twf (trivial win flag) starts at 0 and is set to 1 if the adversary performed an action that amounts to a trivial win.
- mode is \$ when playing IND-ENC\$ or IND-UPD\$ (Figure 7).

Additionally, there are variables that are kept per epoch e.

- k_e is the key of the epoch e.
- Δ_e^i is the update token for the ciphertext with index i to update it to epoch e.
- $\tilde{C}_e$ is the challenge ciphertext (if it has been created yet), updated for the current epoch e.

Sets tracked by the experiment. We define several sets for our experiments (an informal overview is in Section 4.1, Table 3). Several sets ($\mathcal{L}, \tilde{\mathcal{L}}, \mathbb{C}, \mathcal{K}, \mathcal{T}, \mathcal{J}$) are initialized and updated through the code in the experiment's figures (Figures 4 to 6). We refer to them as base sets, and informally describe their role in the model. Then we formally define extended sets, which are base sets together with what an adversary can infer from its corrupted update tokens. We start with corruption related base sets.

- $\mathcal{K}/\mathcal{T}$: Contains the epochs e for which the key/token was leaked via oracle O_{Corr}.
- $\mathbb{C}$: Contains the epochs e for which the current version of the challenge ciphertext was leaked via oracle $O_{\mathsf{Upd}\mathbb{C}}$.

$\mathsf{setup}(\lambda)$
$k_0 \leftarrow \mathsf{UE.Gen}(1^\lambda)$
$e, i \leftarrow 0$
$\mathsf{phase}, \mathsf{twf} \leftarrow 0$
$\mathcal{L}, \tilde{\mathcal{L}}, \mathbb{C}, \mathcal{K}, \mathcal{T}, \mathcal{J} \leftarrow \emptyset$

$O_{\mathsf{Enc}}(M)$
$C \leftarrow \mathsf{UE.Enc}_{k_e}(M)$
$\mathcal{L} \leftarrow \mathcal{L} \cup \{(i, C, e)\}$
$i \leftarrow i + 1$
return C

$O_{\mathsf{Dec}}(C)$
if $\mathsf{phase} = 1 \wedge (C, \cdot) \in \tilde{\mathcal{L}}$
  $\mathsf{twf} \leftarrow 1$
$M \leftarrow \mathsf{UE.Dec}_{k_e}(C)$
return M

O_{Next}
$e \leftarrow e + 1$
$k_e \leftarrow \mathsf{Gen}(1^\lambda)$
for $(i, (\hat{c}, c), e-1) \in \mathcal{L}$
  if $i \notin \mathcal{J}$
    $\Delta_e^i \leftarrow \mathsf{UE.TG}(k_{e-1}, k_e, \hat{c})$
if $\mathsf{phase} = 1$
  if $b = 0 \wedge \mathsf{mode} = \$$
    $\Delta_e^{\mathsf{index}_{\tilde{C}}} \leftarrow\!\!\$ \{0,1\}^{\ell_\Delta}$
  else
    $(\hat{c}, c) \leftarrow \tilde{C}_{e-1}$
    $\Delta_e^{\mathsf{index}_{\tilde{C}}} \leftarrow$
      $\mathsf{UE.TG}(k_{e-1}, k_e, \hat{c})$
  $\tilde{C}_e \leftarrow \mathsf{UE.Upd}(\Delta_e^{\mathsf{index}_{\tilde{C}}}, \tilde{C}_{e-1})$

$O_{\mathsf{Upd}}(C_{e-1})$
if $\nexists j : (j, C_{e-1}, e-1) \in \mathcal{L}$
  return $\perp$
if $j \in \mathcal{J}$: **return** $\perp$
$C_e \leftarrow \mathsf{UE.Upd}(\Delta_e^j, C_{e-1})$
$\mathcal{L} \leftarrow \mathcal{L} \cup \{(j, C_e, e)\}$
return C_e

$O_{\mathsf{Upd}\mathbb{C}}$
$\mathbb{C} \leftarrow \mathbb{C} \cup \{e\}$
$\tilde{\mathcal{L}} \leftarrow \tilde{\mathcal{L}} \cup \{(\tilde{C}_e, e)\}$
return $\tilde{C}_e$

$O_{\mathsf{Try}}(\tilde{C})$
if $e \in \mathcal{K}^*$
  $\mathsf{twf} \leftarrow 1$
if $(\cdot, \tilde{C}, e) \in \mathcal{L}^*$
  $\mathsf{twf} \leftarrow 1$
$M \leftarrow \mathsf{UE.Dec}(k_e, \tilde{C})$
if $M \neq \perp$
  $\mathsf{win} \leftarrow 1$

$O_{\mathsf{Corr}}(\mathsf{inp})$
if $\mathsf{inp} = \mathsf{key}$
  $\mathcal{K} \leftarrow \mathcal{K} \cup \{e\}$
  return k_e
if $\mathsf{inp} = \mathsf{token}$
  $\mathcal{T} \leftarrow \mathcal{T} \cup \{e\}$
  if $\mathsf{phase} = 0$
    return
      $(\Delta_e^i | i \in \{0, 1, \ldots, i-1\})$
  else
    return
      $(\Delta_e^i | i \in \{0, 1, \ldots, i-1\}$
      $\setminus \mathcal{J}), \Delta_e^{\mathsf{index}_{\tilde{C}}}$

Figure 4: Oracles for our security games (c.f. [10], Fig. 6). mode **is set to \$ for our new IND-UPD\$ and IND-ENC\$ notions in Figure 7.**

- $\mathcal{J}$: When the challenge ciphertext is created, the ciphertext(s) that were used to create it are added to $\mathcal{J}$, which means they will not be updated anymore (O_{Next}) and thus cannot have their tokens leaked (O_{Corr}). Instead, the challenge ciphertext ($\mathsf{index}_{\tilde{C}}$) is updated and can have its tokens leaked.

Now we define extended sets for the first three sets. They are initially empty, and updated whenever any oracle is called, by repeating the following assignment code until no further changes occur. (This always terminates as it can only add additional elements from the finite list of all epochs.) Intuitively, $\mathcal{K}^*, \mathcal{T}^*, \mathbb{C}^*$ contain all

Notion	Description	Rationale
s-int-ctxt [10]	Adversary must forge a valid ciphertext (single attempt).	Proof helper for INT-CTXT.
INT-CTXT [10]	Adversary must forge a valid ciphertext.	Proof helper for CCA notions.
xxIND-ENC-atk [10]	Challenger encrypts one of two adversary provided messages.	In addition to providing classic authenticated encryption IND-ENC security, the security must not be lost after updating.
xxIND-UPD-atk [10]	Adversary makes the challenger encrypt and update messages any number of times. Then the challenger updates one of two ciphertexts in the system.	Even if the adversary had full control over the system previously, after a single update the adversary should have no information about the currently stored ciphertexts.

Table 4: Informal overview of security notions, which also exist in similar form in [10].

Notion	Description	Rationale
xxIND-ENC$-atk	Challenger encrypts the (single) adversary provided message, or returns random bits.	As before, now for classic IND-ENC$ security.
xxIND-UPD$-atk	Same as xxIND-UPD-atk, but challenger updates the ciphertext or returns random bits.	After full corruption and one update, the adversary should not be able to distinguish updates from previous ciphertexts from random bits.
ciphertext age hiding	Achieve both xxIND-ENC$-atk and xxIND-UPD$-atk.	Adversary cannot distinguish ciphertexts, even if they have been updated a different number of times (including 0 updates, i.e. fresh encryptions).

Table 5: Informal overview of our novel security notions.

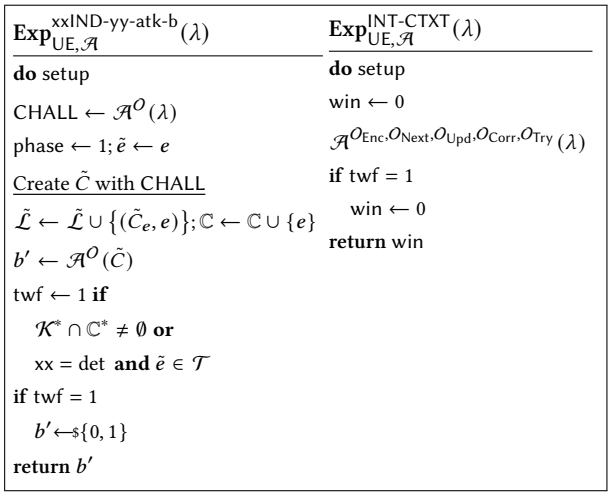

Figure 5: Generic definition of our indistinguishability experiments xxIND-yy-atk **(c.f. [10], Fig. 8), "Create** $\tilde{C}$ **with** CHALL**" is instantiated in Figure 6 or Figure 7.** $O = (O_{\text{Enc}}, O_{\text{Next}}, O_{\text{Upd}}, O_{\text{Upd}\mathbb{C}}, O_{\text{Corr}})$**, and for** atk = CCA**,** O **additionally contains** O_{Dec}**. Also the experiment for the** INT-CTXT **security notion (c.f. [10], Fig. 10).**

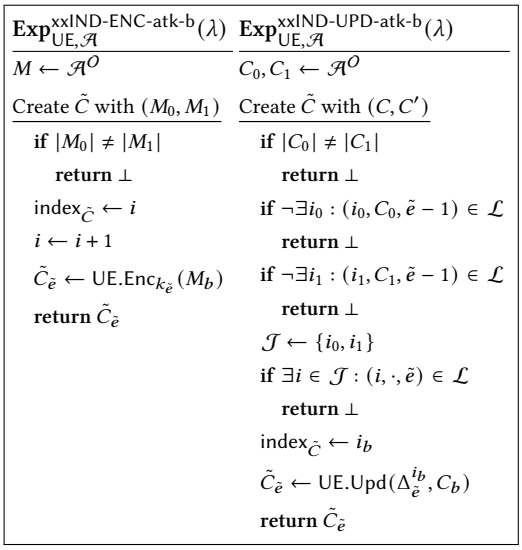

Figure 6: Challenge call definition for xxIND-ENC-atk **and** xxIND-UPD-atk **security experiments. These are instantiations for experiment** xxIND-yy-atk-b **(Figure 5).**

epochs in which the adversary can infer the keys/tokens/ciphertexts. (Our model captures bi-directional updates [22].)

- Define predicate: $\text{inferable}(e, \mathcal{S}, \tilde{\mathcal{T}}) \Leftrightarrow (e-1 \in \mathcal{S} \wedge e \in \tilde{\mathcal{T}}) \vee (e+1 \in \mathcal{S} \wedge e+1 \in \tilde{\mathcal{T}})$.
- $\mathcal{K}^* \leftarrow \{e | e \in \mathcal{K} \vee \text{inferable}(e, \mathcal{K}^*, \mathcal{T})\}$.
- $\mathcal{T}^* \leftarrow \{e | e \in \mathcal{T} \vee (e-1 \in \mathcal{K}^* \wedge e \in \mathcal{K}^*)\}$.
- $\mathbb{C}^* \leftarrow \{e | e \in \mathbb{C} \vee e = \tilde{e} \vee \text{inferable}(e, \mathbb{C}^*, \mathcal{T}^*)\}$.

We define sets to track ciphertexts produced by the experiment. $\mathcal{L}$ contains all regular ciphertexts from oracles ($O_{\text{Enc}}, O_{\text{Upd}}$), while $\tilde{\mathcal{L}}$

holds the challenge ciphertext and its updates from O_{Next}. $\mathcal{J}$ tracks the ciphertext indices of the challenge input(s), preventing trivial wins by disallowing the adversary from seeing update tokens or updated ciphertexts for these indices. Extended sets with * account for what the adversary may infer. The following assignments are repeated until no changes occur.

- $\mathcal{L}$: This set is controlled through code in Figures 4 to 6.
- $\mathcal{L}^* \leftarrow \mathcal{L} \cup \{(\mathsf{index}, \mathsf{UE.Upd}(\Delta_e^{\mathsf{index}}, C), e) | e \in \mathcal{T}^* \wedge (\mathsf{index}, C, e-1) \in \mathcal{L}^*\}$
- $\tilde{\mathcal{L}}$: This set is controlled through code in Figures 4 to 6.
- $\tilde{\mathcal{L}}^* \leftarrow \{(\tilde{C}_e, e) | (\tilde{C}_e, e) \in \tilde{\mathcal{L}} \vee (e \in \mathcal{T}^* \wedge (\tilde{C}_{e-1}, e-1) \in \tilde{\mathcal{L}}^*)\}$.

Definitions. We now formally define security in our model. The experiments are given in Figures 5 and 6.

Definition 4.1. Let UE be an updatable encryption scheme and $\mathcal{A}$ a ppt adversary. For $(\mathsf{xx}, \mathsf{atk}) \in \{(\mathsf{det}, \mathsf{CPA}), (\mathsf{det}, \mathsf{CCA})\}$ we define

$$\mathsf{Adv}^{\mathsf{xxind\text{-}enc\text{-}atk}}_{\mathsf{UE},\mathcal{A}}(\lambda) = \Big|\Pr[\mathbf{Exp}^{\mathsf{xxIND\text{-}ENC\text{-}atk\text{-}1}}_{\mathsf{UE},\mathcal{A}}(\lambda) = 1] - \Pr[\mathbf{Exp}^{\mathsf{xxIND\text{-}ENC\text{-}atk\text{-}0}}_{\mathsf{UE},\mathcal{A}}(\lambda) = 1]\Big|$$

$$\mathsf{Adv}^{\mathsf{xxind\text{-}upd\text{-}atk}}_{\mathsf{UE},\mathcal{A}}(\lambda) = \Big|\Pr[\mathbf{Exp}^{\mathsf{xxIND\text{-}UPD\text{-}atk\text{-}1}}_{\mathsf{UE},\mathcal{A}}(\lambda) = 1] - \Pr[\mathbf{Exp}^{\mathsf{xxIND\text{-}UPD\text{-}atk\text{-}0}}_{\mathsf{UE},\mathcal{A}}(\lambda) = 1]\Big|$$

$$\mathsf{Adv}^{\mathsf{int\text{-}ctxt}}_{\mathsf{UE},\mathcal{A}}(\lambda) = \Pr[\mathbf{Exp}^{\mathsf{INT\text{-}CTXT}}_{\mathsf{UE},\mathcal{A}}(\lambda) = 1].$$

Definition 4.2. Let UE be an updatable encryption scheme. We define the s-INT-CTXT advantage as

$$\mathsf{Adv}^{\mathsf{s\text{-}int\text{-}ctxt}}_{\mathsf{UE},\mathcal{A}}(\lambda) = \Pr[\mathbf{Exp}^{\mathsf{INT\text{-}CTXT}}_{\mathsf{UE},\mathcal{A}} = 1],$$

where $\mathcal{A}$ is ppt and only allowed to make a single query to O_{Try}.

4.3 Comparison to Boyd's Model

Our model in Section 4.2 is largely similar to Boyd's model [10]. The first main difference is considering ciphertext-dependent UE schemes, so instead of just having tokens Δ_e per epoch e, we have tokens Δ_e^i for each ciphertext i per epoch e, which affects O_{Upd}, O_{Next} and O_{Corr}. The second main difference is only modelling chronological adversaries, thus O_{Corr} does not take an epoch index. Finally, we introduce additional experiments in the next section.

4.4 Novel Security Features

In Figure 7 we give novel experiment variants for our IND$ notions, which compare ciphertexts to random strings. We refer to prior notions (IND-ENC and IND-UPD in Figure 6) as Left-Or-Right (LoR) notions because they compare two well-formed ciphertexts. For our new IND-ENC$/IND-UPD$, we model the interactions as follows:

- The adversary provides an input and the challenger returns the real ciphertext (real world) or a random string of the same length (ideal world).
- When generating an update token for the challenge ciphertext, the token generation function TG is not called in the ideal world, and instead a random bitstring is returned. To be precise, in O_{Next} we replace the update tokens $\Delta_e^{\mathsf{index}_{\tilde{C}}}$ for the challenge ciphertext with a random bitstring if phase = 1 (i.e. the challenge ciphertext exists). The security notion thus implies that the scheme having random-looking tokens.
- The update function Upd is called in both worlds. Thus, a secure UE scheme must have an update function which is able to process a random string for both the ciphertext and the token.

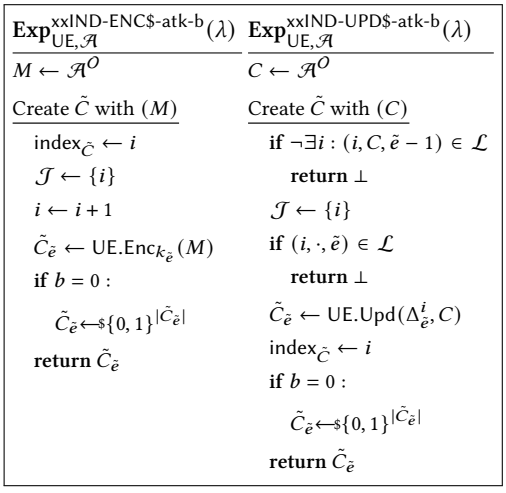

Figure 7: Oracles and instantiations of experiment xxIND-yy-atk **(Figure 5) for our new features.** mode **is set to** $ **for IND-UPD$ and IND-ENC$ notions.**

Definitions. We now define the adversary's advantage for our new IND-ENC$ and IND-UPD$ notions (Figure 7).

Definition 4.3. Let UE be an updatable encryption scheme and $\mathcal{A}$ a ppt adversary. For $(\mathsf{xx}, \mathsf{atk}) \in \{(\mathsf{det}, \mathsf{CPA}), (\mathsf{det}, \mathsf{CCA})\}$ we define

$$\mathsf{Adv}^{\mathsf{xxind\text{-}enc\$\text{-}atk}}_{\mathsf{UE},\mathcal{A}}(\lambda) = \Big|\Pr[\mathbf{Exp}^{\mathsf{xxIND\text{-}ENC\$\text{-}atk\text{-}1}}_{\mathsf{UE},\mathcal{A}}(\lambda) = 1] - \Pr[\mathbf{Exp}^{\mathsf{xxIND\text{-}ENC\$\text{-}atk\text{-}0}}_{\mathsf{UE},\mathcal{A}}(\lambda) = 1]\Big|$$

$$\mathsf{Adv}^{\mathsf{xxind\text{-}upd\$\text{-}atk}}_{\mathsf{UE},\mathcal{A}}(\lambda) = \Big|\Pr[\mathbf{Exp}^{\mathsf{xxIND\text{-}UPD\$\text{-}atk\text{-}1}}_{\mathsf{UE},\mathcal{A}}(\lambda) = 1] - \Pr[\mathbf{Exp}^{\mathsf{xxIND\text{-}UPD\$\text{-}atk\text{-}0}}_{\mathsf{UE},\mathcal{A}}(\lambda) = 1]\Big|$$

Definition 4.4. Let UE be an updatable encryption scheme. Let Δ, c arbitrary. Let $c' \leftarrow \mathsf{UE.Upd}(\Delta, c)$. UE is said to have **length-preserving re-encryptions**, if $\Pr[c' = \perp \vee |c'| = |c|] = 1$.

Definition 4.5. Let UE be an updatable encryption scheme with length-preserving re-encryptions, for which $\mathsf{Adv}^{\mathsf{xxind\text{-}enc\$\text{-}atk}}_{\mathsf{UE},\mathcal{A}}(\lambda)$ and $\mathsf{Adv}^{\mathsf{xxind\text{-}upd\$\text{-}atk}}_{\mathsf{UE},\mathcal{A}}(\lambda)$ are negligible in λ. Then UE is said to be **ciphertext age hiding**.

4.5 Formal Relations between Notions

We show that several classical AE results hold for our UE model, too. In particular, IND$ notions are stronger than the LoR notions. Additionally, a CPA and INT-CTXT secure UE scheme is also CCA secure. We then reduce INT-CTXT to our s-int-ctxt notion. Finally, we show that our new ciphertext age hiding property (which models (A+) from the introduction) implies security corresponding to Boyd's [10] prior formulation of (A+).

Theorem 4.6. *Let* UE *be an updatable encryption scheme and* $\mathcal{A}$ *any ppt adversary. We have*

$$\mathsf{Adv}^{\text{xxind-enc-cpa}}_{\mathsf{UE},\mathcal{A}}(\lambda) \leq 2\mathsf{Adv}^{\text{xxind-enc\$-cpa}}_{\mathsf{UE}}(\lambda),$$
$$\mathsf{Adv}^{\text{xxind-upd-cpa}}_{\mathsf{UE},\mathcal{A}}(\lambda) \leq 2\mathsf{Adv}^{\text{xxind-upd\$-cpa}}_{\mathsf{UE}}(\lambda).$$

Theorem 4.7. *Let* UE *be an updatable encryption scheme. We have*

$$\mathsf{Adv}^{\text{xxind-enc\$-cca}}_{\mathsf{UE}}(\lambda) \leq \mathsf{Adv}^{\text{xxind-enc\$-cpa}}_{\mathsf{UE}}(\lambda) + 2\mathsf{Adv}^{\text{int-ctxt}}_{\mathsf{UE}}(\lambda),$$
$$\mathsf{Adv}^{\text{xxind-upd\$-cca}}_{\mathsf{UE}}(\lambda) \leq \mathsf{Adv}^{\text{xxind-upd\$-cpa}}_{\mathsf{UE}}(\lambda) + 2\mathsf{Adv}^{\text{int-ctxt}}_{\mathsf{UE}}(\lambda).$$

Theorem 4.8. *Let* UE *be an updatable encryption scheme. For any* INT-CTXT *adversary* $\mathcal{A}$ *that makes at most* q_T *queries to* O_{Try} *there is a* s-int-ctxt *adversary* $\mathcal{B}$ *such that*

$$\mathsf{Adv}^{\text{int-ctxt}}_{\mathsf{UE},\mathcal{A}}(\lambda) \leq q_T \mathsf{Adv}^{\text{s-int-ctxt}}_{\mathsf{UE},\mathcal{B}}(\lambda)$$

The proofs are given in Section 4.6. The arguments for Theorem 4.6 are similar to the classical result, that for symmetric encryption indistinguishability-from-random security is stronger than the Left-or-Right variant. The proofs for Theorems 4.7 and 4.8 largely match those of the similar results for Left-or-Right notions in [10].

4.6 Proofs of Formal Relations between Notions

First we show that the indistinguishability from random notions IND-ENC$ and IND-UPD$ are indeed stronger than the Left-or-Right notions (IND-ENC and IND-UPD). This is the case for regular symmetric encryption, but a formal proof for updatable encryption is still required, especially due to ciphertext-specific update tokens.

It is easy to see that there are LoR-secure UE schemes which always return ciphertexts with a fixed structure (e.g. prepended with 10 leading zeroes) and are therefore not indistinguishable from random. We now show that security with respect to indistinguishability from random implies security with respect to Left-or-Right notions.

Proof of Theorem 4.6. First we prove the second inequality using a series of game hops.

- $\mathbf{G_0}$. The game $\mathbf{Exp}^{\text{xxIND-UPD-CPA-0}}_{\mathsf{UE},\mathcal{A}}(\lambda)$.
- $\mathbf{G_1}$. Answer oracle queries using $\mathbf{Exp}^{\text{xxIND-UPD\$-CPA-1}}_{\mathsf{UE},\mathcal{A}}(\lambda)$, but keep track of $\mathcal{L}$ and $\mathcal{J}$, return $\perp$ if $\mathbf{Exp}^{\text{xxIND-UPD-CPA-0}}_{\mathsf{UE},\mathcal{A}}(\lambda)$ would do so due to a check with respect to $\mathcal{L}$ and $\mathcal{J}$ (e.g. for O_{Upd} it checks $i \in \mathcal{J}$ and potentially returns $\perp$). In "Create $\tilde{C}$", query $\mathbf{Exp}^{\text{xxIND-UPD\$-CPA-1}}_{\mathsf{UE},\mathcal{A}}(\lambda)$ with challenge C_0 and put response into $\tilde{C}$. All oracles behave the same in both games, thus this constitutes only a conceptual change, $|\Pr[G_0 \Rightarrow 1] - \Pr[G_1 \Rightarrow 1]| = 0$.
- $\mathbf{G_2}$. Embed $\mathbf{Exp}^{\text{xxIND-UPD\$-CPA-0}}_{\mathsf{UE},\mathcal{A}}(\lambda)$ instead. $|\Pr[G_1 \Rightarrow 1] - \Pr[G_2 \Rightarrow 1]| \leq \mathsf{Adv}^{\text{xxind-upd\$-cpa}}_{\mathsf{UE}}(\lambda)$.
- $\mathbf{G_3}$. In "Create $\tilde{C}$", from $\mathbf{Exp}^{\text{xxIND-UPD-CPA-0}}_{\mathsf{UE},\mathcal{A}}(\lambda)$ input use C_1 instead of C_0. Only a conceptual change, since the ciphertext is still being replaced with random. $|\Pr[G_2 \Rightarrow 1] - \Pr[G_3 \Rightarrow 1]| = 0$.
- $\mathbf{G_4}$. Embed $\mathbf{Exp}^{\text{xxIND-UPD\$-CPA-1}}_{\mathsf{UE},\mathcal{A}}(\lambda)$ again (but still for C_1). $|\Pr[G_3 \Rightarrow 1] - \Pr[G_4 \Rightarrow 1]| \leq \mathsf{Adv}^{\text{xxind-upd\$-cpa}}_{\mathsf{UE}}(\lambda)$.
- $\mathbf{G_5}$. The game $\mathbf{Exp}^{\text{xxIND-UPD-CPA-1}}_{\mathsf{UE},\mathcal{A}}(\lambda)$. Only a conceptual change, $|\Pr[G_4 \Rightarrow 1] - \Pr[G_5 \Rightarrow 1]| = 0$.

Thus $\mathsf{Adv}^{\text{xxind-upd-cpa}}_{\mathsf{UE}}(\lambda) \leq 2\mathsf{Adv}^{\text{xxind-upd\$-cpa}}_{\mathsf{UE}}(\lambda)$. For $\mathsf{Adv}^{\text{xxind-enc\$-cpa}}_{\mathsf{UE}}(\lambda)$ we use the same strategy. □

We now show that a CPA secure scheme which also is secure with respect to INT-CTXT is CCA secure. Similar results are known for regular symmetric encryption and updatable encryption security notions which use a Left-or-Right style [10] (instead of indistinguishability from random).

Proof of Theorem 4.7. As this proof follows the same idea as Theorem 3 in [10], we only give a proof sketch. The proof uses the same game hops for both inequalities.

- $\mathbf{G_0^b}$. The original CCA game with bit b, played by $\mathcal{A}$.

  $\mathsf{Adv}^{\text{xxind-enc\$-cca}}_{\mathsf{UE}}(\lambda) = |\Pr[G_0^0 \Rightarrow 1] - \Pr[G_0^1 \Rightarrow 1]|$

  (or with $\mathsf{Adv}^{\text{xxind-upd\$-cca}}_{\mathsf{UE}}(\lambda)$ for the second inequality).
- $\mathbf{G_1^b}$. Modify O_{Dec} to return $\perp$ if a call is made that would not cause twf $\leftarrow 1$ in an equivalent O_{Try} call, specifically O_{Dec} returns $\perp$ unless $e \in \mathcal{K}^*$ or $(\cdot, C, e) \in \mathcal{L}^*$. This change can only be noticed by an adversary that provides C s.t. the trivial win conditions of O_{Try} do not trigger and the output M is not $\perp$ already, making it a winning adversary in INT-CTXT. $|\Pr[G_0^b \Rightarrow 1] - \Pr[G_1^b \Rightarrow 1]| = \mathsf{Adv}^{\text{int-ctxt}}_{\mathsf{UE}}(\lambda)$.
- $\mathbf{G_2^b}$. Use CPA game to answer oracle queries and challenge, also simulate O_{Dec} by keeping track through other oracles of both $\mathcal{L}$ and the original message m_i for each O_{Enc} call, and returning the trivial query results m_i for $(i, C, e) \in \mathcal{L}^*$ and $\perp$ otherwise. This results in no visible change to the adversary, $|\Pr[G_1^b \Rightarrow 1] - \Pr[G_2^b \Rightarrow 1]| = 0$. G_2 now corresponds to the CPA game, i.e. $\mathsf{Adv}^{\text{xxind-enc\$-cpa}}_{\mathsf{UE}}(\lambda) = |\Pr[G_2^0 \Rightarrow 1] - \Pr[G_2^1 \Rightarrow 1]|$.

Since G_1 modified two games (for $b = 0$ and $b = 1$), the difference between CPA and CCA thus amounts to $2\mathsf{Adv}^{\text{int-ctxt}}_{\mathsf{UE}}(\lambda)$. □

Finally we reduce our INT-CTXT notion (which is needed for the previous theorem) to our s-int-ctxt notion (which is easier to prove for concrete schemes).

Proof of Theorem 4.8. The proof follows a similar idea as Lemma 1 in [10]. $\mathcal{B}$ first randomly draws $i \leftarrow\!\!\$ \{1, \ldots, q_T\}$. Then $\mathcal{B}$ simulates INT-CTXT for $\mathcal{A}$ by using its own oracles, however answers all O_{Try} queries with $\perp$ except the i-th where $\mathcal{B}$ uses its own O_{Try} oracle. As the choice of i was independent, and $\mathcal{A}$ needs to make at least 1 successful query in INT-CTXT to win, $\mathsf{Adv}^{\text{s-int-ctxt}}_{\mathsf{UE},\mathcal{B}}(\lambda) \geq \frac{1}{q_T}\mathsf{Adv}^{\text{int-ctxt}}_{\mathsf{UE}}(\lambda)$. □

4.7 Previous Notions of Ciphertext-Age-Hiding

We introduce the IND-UE experiment (Figure 8) that corresponds to the formulation by Boyd et al. ([10], Definition 8), adapted to our syntax. We show that our ciphertext age hiding notion implies security in this experiment.

Lemma 4.9. *Let* UE *a ciphertext age hiding UE scheme. Then for any ppt adversary* $\mathcal{A}$*,*

$$\Pr[\mathbf{Exp}^{\text{detIND-UE-CCA}}_{\text{UE},\mathcal{A}} = 1] \leq \mathsf{Adv}^{\text{detind-enc\$-cca}}_{\text{UE}}(\lambda) + \mathsf{Adv}^{\text{detind-upd\$-cca}}_{\text{UE}}(\lambda),$$

which is negligible.

Proof. We use a series of game hops.

(1) Game G_1: $\mathbf{Exp}^{\text{detIND-UE-CCA-0}}_{\text{UE},\mathcal{A}}$ (Figure 8).

(2) Game G_2: After the line $\tilde{C}_{\tilde{e}} \leftarrow \mathsf{UE.Upd}(\Delta^{i_C}_{\tilde{e}}, C)$, add $\tilde{C}_{\tilde{e}} \leftarrow\!\!\$ \{0,1\}^{|\tilde{C}_{\tilde{e}}|}$ and set mode $\leftarrow$ \$. Now G_0 and G_1 match $\mathbf{Exp}^{\text{detIND-UPD\$-CCA-1}}_{\text{UE},\mathcal{A}}$ and $\mathbf{Exp}^{\text{detIND-UPD\$-CCA-0}}_{\text{UE},\mathcal{A}}$ (Figure 7), except that adversary is additionally restrained through the first 4 lines in "Create C" of Figure 8 (the additional element in $\mathcal{J}$ removes a token from potential calls to O_{Corr}). Thus

$$|\Pr[G_1 \Rightarrow 1] - \Pr[G_2 \Rightarrow 1]| \leq \mathsf{Adv}^{\text{detind-upd\$-cca}}_{\text{UE}}(\lambda).$$

(3) Game G_3: Replace $\tilde{C}_{\tilde{e}} \leftarrow\!\!\$ \{0,1\}^{|\tilde{C}_{\tilde{e}}|}$ with $\tilde{C}_{\tilde{e}} \leftarrow\!\!\$ \{0,1\}^{\ell_C(M)}$. From correctness, $\tilde{C}_{\tilde{e}} \neq \perp$ and due to having length-preserving updates (from ciphertext age hiding), $\tilde{C}_{\tilde{e}}$ still has size $\ell_C(M)$ after updating. $|\Pr[G_2 \Rightarrow 1] - \Pr[G_3 \Rightarrow 1]| = 0$.

(4) Game G_4: $\mathbf{Exp}^{\text{detIND-UE-CCA-1}}_{\text{UE},\mathcal{A}}$ (Figure 8). Since G_3 and G_4 match $\mathbf{Exp}^{\text{detIND-ENC\$-CCA-0}}_{\text{UE},\mathcal{A}}$ and $\mathbf{Exp}^{\text{detIND-ENC\$-CCA-1}}_{\text{UE},\mathcal{A}}$ with the additional constraints,

$$|\Pr[G_3 \Rightarrow 1] - \Pr[G_4 \Rightarrow 1]| \leq \mathsf{Adv}^{\text{detind-enc\$-cca}}_{\text{UE}}(\lambda).$$

The overall advantage is negligible due to Definition 4.5. □

$\mathbf{Exp}^{\text{detIND-UE-CCA-}b}_{\text{UE},\mathcal{A},\mathcal{A}}$

$M, C \leftarrow \mathcal{A}^{O}$

Create $\tilde{C}$ with (M, C)

- **if** $\ell_C(M) \neq |C|$: **return** $\perp$
- **if** $\neg\exists i_C : (i_C, C, \tilde{e} - 1) \in \mathcal{L}$: **return** $\perp$
- **if** $(i, \cdot, \tilde{e}) \in \mathcal{L}$: **return** $\perp$
- $\mathcal{J} \leftarrow \{i_C, i\}$
- **if** $b = 0$
  - $\tilde{C}_{\tilde{e}} \leftarrow \mathsf{UE.Enc}_{k_{\tilde{e}}}(M)$
  - $\mathsf{index}_{\tilde{C}} \leftarrow i$
- **else**
  - $\tilde{C}_{\tilde{e}} \leftarrow \mathsf{UE.Upd}(\Delta^{i_C}_{\tilde{e}}, C)$
  - $\mathsf{index}_{\tilde{C}} \leftarrow i_C$
- $i \leftarrow i + 1$
- **return** $\tilde{C}_{\tilde{e}}$

Figure 8: Challenge call definition for the detIND-UE-CCA **security experiment, as an instantiation for experiment** xxIND-yy-atk-b **(Figure 5).**

4.8 Model summary

To achieve full security in our model, schemes should achieve IND-ENC$-CPA, IND-UPD$-CPA and s-int-ctxt and have length-preserving updates. These imply the other notions (IND-ENC$-CCA, IND-UPD$-CCA, ciphertext age hiding) and those corresponding to prior models [10] (IND-ENC-CCA, IND-UPD-CCA, IND-UE-CCA).

5 Constructions

We present our schemes PUE-One and PUE-List in Figures 10 and 11. All our algorithms immediately return $\perp$ if an internal Dec call returns $\perp$.

5.1 Design rationale for our schemes

Overall, PUE-One is designed for relaxed security assumptions (no token corruptions) whereas PUE-List targets full security in our model.

Design Rationale of PUE-One**.** As a starting point for designing an efficient UE scheme, consider the simple UE scheme called hybrid encryption [8, 15] (HES), which is also used in a similar form in industry (Amazon AWS [1] and Google [18]). There, ciphertexts are

$$(\hat{c}, c) = (\mathsf{AE.Enc}_k(k_{\text{Data}}), \mathsf{AE.Enc}_{k_{\text{Data}}}(m))$$

where k is the key of the scheme and k_{Data} is freshly sampled whenever encrypting a message (m in the equation above). Encryptions are very fast, as they use an authenticated encryption scheme AE on the message, the only overhead is encrypting the constant size k_{Data}. The scheme allows efficient updates; to replace k by k', only the constant size ciphertext header $\hat{c}$ needs to be replaced by $\mathsf{AE.Enc}_{k'}(k_{\text{Data}})$. The update token that the client sends to the server consists exactly of this new $\hat{c}$. Since c is not changed, this scheme cannot achieve ciphertext unlinkability (IND-UPD); an adversary can trivially compare the c part of the old ciphertext with c of the updated ciphertext.

The core idea of our constructions is re-randomizing c through a PRG G. This idea has not been explored in prior literature. The PRG seed is sampled during the updating procedure and stored in $\hat{c}$. As a first attempt, consider structuring the ciphertexts as

$$(\hat{c}, c) = (\mathsf{AE.Enc}_k(k_{\text{Data}}, \mathsf{seed}), \mathsf{AE.Enc}_{k_{\text{Data}}}(m) \oplus G(\mathsf{seed}))$$

i.e. the encrypted m is masked with some PRG output. To update, we replace the constant size $\hat{c}$ with $\mathsf{AE.Enc}_{k'}(k_{\text{Data}}, \mathsf{seed}')$ for some new k' and seed'. Additionally, c is updated by XORing with $G(\mathsf{seed})$ (to remove the old mask) and with $G(\mathsf{seed}')$ (to apply the new mask). The update token therefore contains both seed and seed' in plain. To decrypt, the mask is removed by XORing with $G(\mathsf{seed})$ and then decrypted as for HES. Unfortunately, this scheme attempt allows ciphertext forgeries, i.e. it is not INT-CTXT secure. The reason is that $\hat{c}$ is not binding to c, which allows an attacker to keep $\hat{c}$ of an existing ciphertext while replacing c. AE authenticity only gives security guarantees if the keys k, k_{Data} are secret, but in our model they may have been obtained in a prior epoch. We detail this attack in Section 5.2.

To fix the problem, for PUE-One we structure the ciphertext as

$$(\hat{c}, c) = (\mathsf{AE.Enc}_k(H(c_0), k_{\text{Data}}, \mathsf{seed}), c_0 \oplus G(\mathsf{seed}))$$

where $c_0 := \mathsf{SE.Enc}_{k_{\mathsf{Data}}}(m)$. A collision resistant hash H binds $\hat{c}$ to c_0, even when the key is revealed. This $H(c_0)$ in $\hat{c}$ also ensures the authenticity of c, which allows us to use an (unauthenticated) encryption scheme SE rather than AE and thereby improve overall efficiency. To further optimize encryption efficiency, newly encrypted ciphertexts are not masked with a PRG output. Instead they are of the form

$$(\hat{c}, c) = (\mathsf{AE.Enc}_k(H(c_0), k_{\mathsf{Data}}, 0), c_0)$$

where we reserve the value seed = 0 to indicate that no mask was applied. It is necessary that we do not simply leave the seed out of the newly encrypted $\hat{c}$, as this would change the length of $\hat{c}$ and thereby reveal the age of the ciphertext. We give PUE-One in Figure 10 and an exemplary execution of PUE-One in Figure 9.

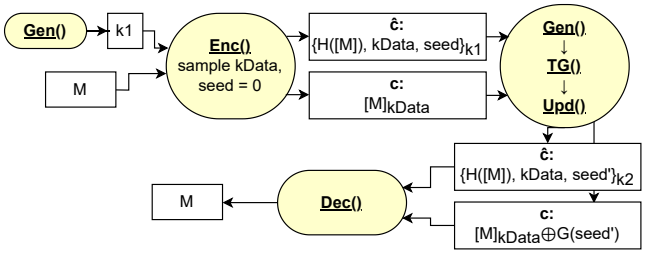

Figure 9: PUE-One **message flow.** [.] **denotes symmetric encryption,** {.} **denotes authenticated encryption.**

$\mathsf{Gen}()$

$k \leftarrow \mathsf{AE.Gen}()$

return k

$\mathsf{Enc}_k(m)$

$k_{\mathsf{Data}} \leftarrow \mathsf{SE.Gen}()$

$c \leftarrow \mathsf{SE.Enc}_{k_{\mathsf{Data}}}(m)$

$h \leftarrow \mathsf{H}(c)$

$\mathsf{seed} \leftarrow 0^z$

$\hat{c} \leftarrow \mathsf{AE.Enc}_k(h, k_{\mathsf{Data}}, \mathsf{seed})$

return $(\hat{c}, c)$

$\mathsf{Dec}_k((\hat{c}, c))$

$h, k_{\mathsf{Data}}, \mathsf{seed} \leftarrow \mathsf{AE.Dec}_k(\hat{c})$

if $\mathsf{seed} \neq 0^z$

  $c \leftarrow c \oplus G(\mathsf{seed})[0, |c|]$

if $h \neq \mathsf{H}(c)$: **return** $\perp$

$m \leftarrow \mathsf{SE.Dec}_{k_{\mathsf{Data}}}(c)$

return m

$\mathsf{TG}(k_{\mathsf{old}}, k_{\mathsf{new}}, \hat{c})$

$h, k_{\mathsf{Data}}, \mathsf{seed} \leftarrow \mathsf{AE.Dec}_{k_{\mathsf{old}}}(\hat{c})$

$\mathsf{seed}' \leftarrow\!\!\$ \{0,1\}^z \setminus \{0^z\}$

$\hat{c}' \leftarrow \mathsf{AE.Enc}_{k_{\mathsf{new}}}(h, k_{\mathsf{Data}}, \mathsf{seed}')$

$\Delta \leftarrow (\hat{c}', \mathsf{seed}, \mathsf{seed}')$

return Δ

$\mathsf{Upd}(\Delta, (\hat{c}, c))$

$(\hat{c}', \mathsf{seed}, \mathsf{seed}') \leftarrow \Delta$

if $\mathsf{seed} \neq 0^z$

  $c \leftarrow c \oplus G(\mathsf{seed})[0, |c|]$

$c \leftarrow c \oplus G(\mathsf{seed}')[0, |c|]$

return $(\hat{c}', c)$

Figure 10: Scheme PUE-One**, based on an** AE**, a symmetric encryption scheme** SE**, a collision resistant hash** H **and a PRG** G **with** z **bit seeds.**

Design Rationale of PUE-List**.** Our scheme PUE-List (Figure 11) is a variant of PUE-One. PUE-One provides no unlinkability (IND-UPD) if the tokens are leaked. The problem is that at any point in time, the current token alone allows to remove the current $G(\mathsf{seed})$ mask from a ciphertext, leaving only the c from the initial ciphertext before any updates.

$\mathsf{Gen}()$ | $\mathsf{Enc}_k(m)$

$k \leftarrow \mathsf{AE.Gen}()$ | $k_{\mathsf{Data}} \leftarrow \mathsf{SE.Gen}()$

return k | $S \leftarrow \emptyset$

$c \leftarrow \mathsf{SE.Enc}_{k_{\mathsf{Data}}}(m)$

$\mathsf{seed}_{\mathsf{Token}} \leftarrow\!\!\$ \{0,1\}^z$

$h \leftarrow \mathsf{H}(c)$

$\hat{c}_{\mathsf{main}} \leftarrow \mathsf{AE.Enc}_k(h, k_{\mathsf{Data}}, \mathsf{seed}_{\mathsf{Token}}, \langle S \rangle_{\mathsf{p}})$

$\hat{c} \leftarrow (\mathsf{seed}_{\mathsf{Token}}, \hat{c}_{\mathsf{main}})$

return $(\hat{c}, c)$

$\mathsf{Dec}_k((\hat{c}, c))$

$\mathsf{seed}_{\mathsf{Token}}, \hat{c}_{\mathsf{main}} \leftarrow \hat{c}$

$h, k_{\mathsf{Data}}, \mathsf{seed}_{\mathsf{Token}}', S \leftarrow \mathsf{AE.Dec}_k(\hat{c}_{\mathsf{main}})$

if $\mathsf{seed}_{\mathsf{Token}} \neq \mathsf{seed}_{\mathsf{Token}}'$: **return** $\perp$

for $\mathsf{seed} \in S$: $c \leftarrow c \oplus G(\mathsf{seed})[0, |c|]$

if $h \neq \mathsf{H}(c)$: **return** $\perp$

$m \leftarrow \mathsf{SE.Dec}_{k_{\mathsf{Data}}}(c)$

return m

$\mathsf{TG}(k_{\mathsf{old}}, k_{\mathsf{new}}, \hat{c})$

$\mathsf{seed}_{\mathsf{Token}}, \hat{c}_{\mathsf{main}} \leftarrow \hat{c}$

$h, k_{\mathsf{Data}}, \mathsf{seed}_{\mathsf{Token}2}, S \leftarrow \mathsf{AE.Dec}_{k_{\mathsf{old}}}(\hat{c}_{\mathsf{main}})$

if $\mathsf{seed}_{\mathsf{Token}} \neq \mathsf{seed}_{\mathsf{Token}2} \vee |S| = \mathsf{p}$: **return** $\perp$

$\mathsf{seed} \leftarrow\!\!\$ \{0,1\}^z; S \leftarrow S \cup \{\mathsf{seed}\}$

$\mathsf{seed}_{\mathsf{Token}}' \leftarrow\!\!\$ \{0,1\}^z$

$\hat{c}'_{\mathsf{main}} \leftarrow \mathsf{AE.Enc}_{k_{\mathsf{new}}}(h, k_{\mathsf{Data}}, \mathsf{seed}_{\mathsf{Token}}', \langle S \rangle_{\mathsf{p}})$

$\hat{c}' \leftarrow (\mathsf{seed}_{\mathsf{Token}}', \hat{c}'_{\mathsf{main}}); \Delta' \leftarrow (\hat{c}', \mathsf{seed})$

$\Delta \leftarrow \Delta' \oplus G(\mathsf{seed}_{\mathsf{Token}})[0, |\Delta'|]$

return Δ

$\mathsf{Upd}(\Delta, (\hat{c}, c))$

$\mathsf{seed}_{\mathsf{Token}}, \hat{c}_{\mathsf{main}} \leftarrow \hat{c}$

$\Delta' \leftarrow \Delta \oplus G(\mathsf{seed}_{\mathsf{Token}})[0, |\Delta|]$

$(\hat{c}', \mathsf{seed}) \leftarrow \Delta'$

$c \leftarrow c \oplus G(\mathsf{seed})[0, |c|]$

return $(\hat{c}', c)$

Figure 11: Scheme PUE-List**, based on an** AE**, a CPA-secure symmetric encryption scheme** SE**, collision resistant hash** H **and a PRG** G **with** z **bit seeds. Parameter** $\mathsf{p} \geq 1$ **gives the maximum number of updates.**

Step 1. To fix this, consider having ciphertexts with the structure

$$(\hat{c}, c) = (\mathsf{AE.Enc}_k(k_{\mathsf{Data}}, S, H(c)), \mathsf{SE.Enc}_{k_{\mathsf{Data}}}(m) \oplus G(\mathsf{seed}_1) \ldots \oplus G(\mathsf{seed}_\ell))$$

where S is the list of all seeds $(\mathsf{seed}_1, \ldots, \mathsf{seed}_\ell)$. Instead of removing the old $G(\mathsf{seed})$ mask during updating, we keep adding more layers to the ciphertext. Thus for an attacker to demask some ciphertext,

all tokens are needed which amounts to a trivial win anyways. To decrypt, one must remove each layer through $\oplus G(\text{seed})$ for all seed $\in S$ and then use SE / AE decryption. Thus decryptions slow down with the number of ciphertext updates, yet our benchmarks show better decryption performance than asymmetric UE schemes for practical numbers of updates due to the efficiency of PRGs. To keep ciphertext length constant, S is padded to a predefined size, limiting the maximum number of updates in PUE-List.

Step 2. To ensure full security, we must keep S secret from the adversary, as it contains all prior update tokens. Our IND-UPD games allow k and token Δ corruption (which contains S) but prevent revealing the ciphertext in the same epoch. We therefore mask the token with $G(\text{seed}_{\mathsf{Token}})$. Since the server needs access to tokens, $\text{seed}_{\mathsf{Token}}$ is stored in plain next to the ciphertext and replaced with each update. Thus the ciphertext always contains $\text{seed}_{\mathsf{Token}}$ to demask the next token, which carries the next $\text{seed}_{\mathsf{Token}}{}'$. Consequently, tokens are useless to the adversary unless it corrupts the ciphertext simultaneously. To access S within the token, the adversary additionally needs the key, but corrupting the key, ciphertext and token triggers a trivial win condition.

5.2 Attack against the hypothetical scheme

In Section 5.1 we give a hypothetical scheme with ciphertext structure

$$(\hat{c}, c) = (\mathsf{AE.Enc}_k(k_{\mathsf{Data}}, \text{seed}), \mathsf{AE.Enc}_{k_{\mathsf{Data}}}(m) \oplus G(\text{seed}))$$

To forge a ciphertext in the current epoch, an attacker needs the key of a prior epoch and any ciphertext. In more detail, first take an arbitrary ciphertext $(\hat{c}, c)$, corrupt key k, decrypt $\hat{c}$ to obtain k_{Data}, seed, and let $\bar{c} \leftarrow c \oplus G(\text{seed})$ the AE ciphertext of the data. Now compute $\bar{c'} \leftarrow \mathsf{AE.Enc}_{k_{\mathsf{Data}}}(m')$ for an arbitrary m', s.t. $|\bar{c}| = |\bar{c'}|$, and let $B = \bar{c} \oplus \bar{c'}$. Observe that $(\hat{c}, c \oplus B)$ is now a valid ciphertext that decrypts to m'. It is not enough to win the INT-CTXT game just yet, as we revealed the current key and thereby would trigger the trivial win condition. Instead, we need to use O_{Next} to go to a new epoch, and obtain the new version of the ciphertext, say $(\hat{c}_2, c_2)$, and give the forgery $(\hat{c}_2, c_2 \oplus B)$ for the current epoch.

6 Security

Let $\mathsf{e}_{\max}$ denote the total number of epochs during the experiment. The correctness of our schemes is in Section 6.1 and their security in Section 6.2. All security proofs are in Section 6.3.

6.1 Correctness of our schemes

Lemma 6.1. *Our scheme* PUE-One *is correct.* PUE-List *is correct under the assumption that ciphertexts are not updated more often than the specified parameter* p.

Proof. From inspecting the Enc functions, it is clear that ciphertexts initially follow the form that is described in Section 5.1. From inspecting Upd, note that correctly formed ciphertexts remain in this structure. Inspecting Dec reveals that ciphertexts of this form return the initial m. □

6.2 Security

The following theorems establish the security of our schemes. The factor $\mathsf{e}_{\max}{}^3$ is a standard loss in this setting, also seen in SHINE schemes by Boyd et al. ([10], Theorem 4). Similarly, the number of decryption queries q_D appears in terms related to adversarial ciphertext forgery. In our schemes, AE authenticity prevents forgery of ciphertext headers $\hat{c}$, while the collision resistance of H ensures that $H(c)$ in $\hat{c}$ binds to the correct c. The remaining bound reflects the advantage against standard security notions for all scheme components. First we give PUE-List security.

Theorem 6.2. *For any ppt adversary $\mathcal{A}$ using q_D O_{Dec} queries,*

$$\begin{aligned}\mathsf{Adv}^{\text{detind-enc\$-cca}}_{\mathsf{PUE\text{-}List},\mathcal{A}}(\lambda) &\le 2\mathsf{e}_{\max}{}^3\mathsf{Adv}^{\text{priv}}_{\mathsf{AE}}(\lambda)\\ &+ \mathsf{e}_{\max}{}^2\mathsf{Adv}^{\text{prg}}_{G}(\lambda,\mathsf{y}) + \mathsf{e}_{\max}{}^2\mathsf{Adv}^{\text{priv}}_{\mathsf{SE}}(\lambda)\\ &+ 2q_D\mathsf{e}_{\max}\mathsf{Adv}^{\text{auth}}_{\mathsf{AE}}(\lambda) + 2q_D\mathsf{e}_{\max}\mathsf{Adv}^{\text{collres}}_{H}.\end{aligned}$$

Theorem 6.3. *For any ppt adversary $\mathcal{A}$ using q_D O_{Dec} queries and, in case of* PUE-List *causes at most $\mathsf{e}_{\max} \le \mathsf{p}$ epochs we have*

$$\begin{aligned}&\mathsf{Adv}^{\text{detind-upd\$-cca}}_{\mathsf{PUE\text{-}List},\mathcal{A}}(\lambda) \le 2\mathsf{e}_{\max}{}^3\mathsf{Adv}^{\text{priv}}_{\mathsf{AE}}(\lambda)\\ &+ 2\mathsf{e}_{\max}{}^2\mathsf{Adv}^{\text{prg}}_{G}(\lambda,\mathsf{y}) + 2q_D\mathsf{e}_{\max}\mathsf{Adv}^{\text{auth}}_{\mathsf{AE}}(\lambda) + 2q_D\mathsf{e}_{\max}\mathsf{Adv}^{\text{collres}}_{H}.\end{aligned}$$

Now we give PUE-One security.

Theorem 6.4. *For any ppt adversary $\mathcal{A}$ using q_D O_{Dec} queries,*

$$\begin{aligned}&\mathsf{Adv}^{\text{detind-enc-cca}}_{\mathsf{PUE\text{-}One},\mathcal{A}}(\lambda) \le 4\mathsf{e}_{\max}{}^3\mathsf{Adv}^{\text{priv}}_{\mathsf{AE}}(\lambda) + 2\mathsf{e}_{\max}{}^2\mathsf{Adv}^{\text{prg}}_{G}(\lambda,\mathsf{y})\\ &+ 2\mathsf{e}_{\max}{}^2\mathsf{Adv}^{\text{priv}}_{\mathsf{SE}}(\lambda) + 2q_D\mathsf{e}_{\max}\mathsf{Adv}^{\text{auth}}_{\mathsf{AE}}(\lambda) + 2q_D\mathsf{e}_{\max}\mathsf{Adv}^{\text{collres}}_{H}.\end{aligned}$$

If $\mathcal{A}$ additionally never uses O_{Corr} with inp = token*,*

$$\begin{aligned}&\mathsf{Adv}^{\text{detind-enc\$-cca}}_{\mathsf{PUE\text{-}One},\mathcal{A}}(\lambda) \le 2\mathsf{e}_{\max}{}^3\mathsf{Adv}^{\text{priv}}_{\mathsf{AE}}(\lambda) + \mathsf{e}_{\max}{}^2\mathsf{Adv}^{\text{prg}}_{G}(\lambda,\mathsf{y})\\ &+ \mathsf{e}_{\max}{}^2\mathsf{Adv}^{\text{priv}}_{\mathsf{SE}}(\lambda) + 2q_D\mathsf{e}_{\max}\mathsf{Adv}^{\text{auth}}_{\mathsf{AE}}(\lambda) + 2q_D\mathsf{e}_{\max}\mathsf{Adv}^{\text{collres}}_{H}.\end{aligned}$$

Theorem 6.5. *For any ppt adversary $\mathcal{A}$ that makes q_D O_{Dec} queries and never uses O_{Corr} with* inp = token*,*

$$\begin{aligned}\mathsf{Adv}^{\text{detind-upd\$-cca}}_{\mathsf{PUE\text{-}One},\mathcal{A}}(\lambda) &\le 2\mathsf{e}_{\max}{}^3\mathsf{Adv}^{\text{priv}}_{\mathsf{AE}}(\lambda) + \mathsf{e}_{\max}{}^2\mathsf{Adv}^{\text{prg}}_{G}(\lambda,\mathsf{y})\\ &+ 2q_D\mathsf{e}_{\max}\mathsf{Adv}^{\text{auth}}_{\mathsf{AE}}(\lambda) + 2q_D\mathsf{e}_{\max}\mathsf{Adv}^{\text{collres}}_{H}.\end{aligned}$$

6.3 Proofs

Our general proof strategy is to prove CPA security, INT-CTXT security and then use Theorem 4.7 (CPA + INT-CTXT ⇒ CCA).

INT-CTXT Security. We start proving the security of our schemes by showing they achieve INT-CTXT security.

Lemma 6.6. *For any ppt adversary $\mathcal{A}$ using q_T O_{Try} queries,*

$$\begin{aligned}\mathsf{Adv}^{\text{int-ctxt}}_{\mathsf{PUE\text{-}List},\mathcal{A}}(\lambda) &\le q_T\mathsf{e}_{\max}(\mathsf{Adv}^{\text{auth}}_{\mathsf{AE}}(\lambda) + \mathsf{Adv}^{\text{collres}}_{H}),\\ \mathsf{Adv}^{\text{int-ctxt}}_{\mathsf{PUE\text{-}One},\mathcal{A}}(\lambda) &\le q_T\mathsf{e}_{\max}(\mathsf{Adv}^{\text{auth}}_{\mathsf{AE}}(\lambda) + \mathsf{Adv}^{\text{collres}}_{H}).\end{aligned}$$

Proof. We start from the s-int-ctxt game. The final bound thus incurs an additional factor q_T (Theorem 4.8).

Game hops.

- G_0: Game s-int-ctxt, played by $\mathcal{A}$.
- G_1: Guess the epoch $\tilde{e}$ in which $\mathcal{A}$ calls O_{Try}, abort if wrong. $\Pr[G_1 \Rightarrow 1] \ge \frac{1}{\mathsf{e}_{\max}} \Pr[G_0 \Rightarrow 1]$.

$\mathbf{Exp}^{\text{DIC-}b}_{\text{AE},\mathcal{A}}$	$\mathcal{E}(M)$	$O_{\text{Corr}}()$
twf $\leftarrow$ false	$C_1 \leftarrow \text{AE.Enc}_k(M)$	twf $\leftarrow$ true
$k \leftarrow \text{AE.Gen}()$	$C_0 \leftarrow\!\!\$ \{0,1\}^{\lvert C_1\rvert}$	**return** k
$b' \leftarrow \mathcal{A}^{\text{AE.Enc}_k(\cdot),\mathcal{E}(\cdot),O_{\text{Corr}}}$	**return** C_b	
if twf = true **then return** 0		
return b		

Figure 12: Experiment DIC **(dual IND-CPA$ with corruption)**

- G_2: In epoch $\tilde{e}$, if O_{Try} is called with a $\hat{c}$ that wasn't generated by the experiment for the current epoch (i.e. $(\cdot,(\hat{c},\cdot),e) \notin \mathcal{L}^*$), return $\perp$. This change is only noticeable if the adversary was able to give such a dishonest $\hat{c}$ without AE.Dec returning $\perp$. $\Pr[G_2 \Rightarrow 1] \geq \Pr[G_1 \Rightarrow 1] - \text{Adv}^{\text{auth}}_{\text{AE}}(\lambda)$.

In order to win in G_2, $\mathcal{A}$ must call O_{Try} on a ciphertext $(\hat{c},c)$ where $\hat{c}$ was produced by an oracle $((\cdot,(\hat{c},\cdot),e) \in \mathcal{L}^*$ from G_2 definition) and c is "new" $((\cdot,(\hat{c},c),e) \notin \mathcal{L}^*$ from $\mathbf{Exp}^{\text{INT-CTXT}}_{\text{PUE-List},\mathcal{A}}$ definition). This implies finding a collision in the hash value: We define $\mathcal{B}$ against collRes of H. $\mathcal{B}$ runs G_2, and when $\mathcal{A}$ calls O_{Try} with ciphertext $(\hat{c},c)$ $\mathcal{B}$ outputs $x_0 = c$ and for x_1 the (first) ciphertext c' with $(\cdot,(\hat{c},c'),e) \in \mathcal{L}^*$ (such a c' must exist due to the previous consideration). Thus $\Pr[G_2 \Rightarrow 1] \leq \text{Adv}^{\text{collres}}_{H}$. □

IND-CPA($) Security. We define a new security experiment DIC (Figure 12) which we prove equivalent to the priv game for AE. Yet, DIC is easier to embed later in the proof of Lemma 6.9.

Definition 6.7. Let AE an AE scheme. We define the DIC (Figure 12) advantage for any ppt adversary $\mathcal{A}$ as

$$\text{Adv}^{\text{dic}}_{\text{AE},\mathcal{A}}(\lambda) = \left|\Pr[\mathbf{Exp}^{\text{DIC-0}}_{\text{AE},\mathcal{A}} = 1] - \Pr[\mathbf{Exp}^{\text{DIC-1}}_{\text{AE},\mathcal{A}} = 1]\right|$$

Lemma 6.8. *For any ppt adversary $\mathcal{A}$,*

$$\text{Adv}^{\text{dic}}_{\text{AE},\mathcal{A}}(\lambda) \leq 2\text{Adv}^{\text{priv}}_{\text{AE}}(\lambda).$$

Proof Sketch. We prove Lemma 6.8 using a series of games.

- G_0: The original DIC game for $b = 0$.
- G_1: O_{Corr} always returns $\perp$. Note that for any adversary in G_0, we can construct a G_0 adversary with equivalent advantage that returns 0 instead of calling O_{Corr}. Thus without loss of generality we assume $\mathcal{A}$ never calls O_{Corr}. $|\Pr[G_0 = 1] - \Pr[G_1 = 1]| = 0$
- G_2: The AE.Enc_k oracle of the adversary is replaced with a \$ oracle (returning random strings of the respective ciphertext's length). We have $|\Pr[G_1 = 1] - \Pr[G_2 = 1]| \leq \text{Adv}^{\text{priv}}_{\text{AE}}(\lambda)$. To see this, create an adversary against PRIV of AE and use the provided oracle in place of the first oracle, which matches Enc_k of G_1 or \$ of G_2.
- G_3: Both of the adversary's oracles are replaced with Enc_k. $|\Pr[G_2 = 1] - \Pr[G_3 = 1]| \leq \text{Adv}^{\text{priv}}_{\text{AE}}(\lambda)$. In the reduction we use the oracle from the PRIV challenger for both of the encryption oracles given to $\mathcal{A}$.
- G_4: Restore the original functionality of O_{Corr}. As for G_1 transition, $|\Pr[G_3 = 1] - \Pr[G_4 = 1]| = 0$. This game is identical to the original DIC game for $b = 1$.

□

We turn to the CPA security of our schemes and utilize our DIC game in the proof. This then finalizes our proof of Theorems 6.2 to 6.5 by utilizing Theorem 4.7 (on achieving CCA from CPA + INT-CTXT) and Lemma 6.6.

Lemma 6.9. *For any ppt adversary $\mathcal{A}$ we have*

$$\text{Adv}^{\text{detind-enc\$-cpa}}_{\text{PUE-List},\mathcal{A}}(\lambda) \leq \mathsf{e}_{\max}{}^2(2\mathsf{e}_{\max}\text{Adv}^{\text{priv}}_{\text{AE}}(\lambda) + \text{Adv}^{\text{prg}}_{G}(\lambda,\mathsf{y}) + \text{Adv}^{\text{priv}}_{\text{SE}}(\lambda)).$$

Let p *the parameter of* PUE-List. *For any ppt adversary $\mathcal{A}$ that causes at most* $\mathsf{e}_{\max} \leq \mathsf{p}$ *epochs we have*

$$\text{Adv}^{\text{detind-upd\$-cpa}}_{\text{PUE-List},\mathcal{A}}(\lambda) \leq \mathsf{e}_{\max}{}^2(2\mathsf{e}_{\max}\text{Adv}^{\text{priv}}_{\text{AE}}(\lambda) + 2\text{Adv}^{\text{prg}}_{G}(\lambda,\mathsf{y}))$$

For any ppt adversary $\mathcal{A}$ that never uses O_{Corr} with inp = token,

$$\text{Adv}^{\text{detind-enc\$-cpa}}_{\text{PUE-One}}(\lambda) \leq \mathsf{e}_{\max}{}^2(2\mathsf{e}_{\max}\text{Adv}^{\text{priv}}_{\text{AE}}(\lambda) + \text{Adv}^{\text{prg}}_{G}(\lambda,\mathsf{y}) + \text{Adv}^{\text{priv}}_{\text{SE}}(\lambda)),$$
$$\text{Adv}^{\text{detind-upd\$-cpa}}_{\text{PUE-One},\mathcal{A}}(\lambda) \leq \mathsf{e}_{\max}{}^2(2\mathsf{e}_{\max}\text{Adv}^{\text{priv}}_{\text{AE}}(\lambda) + \text{Adv}^{\text{prg}}_{G}(\lambda,\mathsf{y})).$$

Proof of Lemma 6.9. We describe a series of games which are applicable for bounding all of the advantage terms. The few differences for the different experiments will be pointed out when they appear.

- G_0: Original game for $b = 1$.
- G_1: Guess challenge epoch $\tilde{e}$, in which the adversary ends phase 0 and returns the message or ciphertext to the experiment. Abort if this guess was wrong. This results in $\Pr[G_1 \Rightarrow 1] \geq \frac{1}{\mathsf{e}_{\max}} \Pr[G_0 \Rightarrow 1]$.
- G_2: *Intuition: If* $\text{seed}_{\text{Token}}$ *is not known, then we use the PRG's security to randomize the tokens.* In this game, the experiment tracks an additional variable $\text{know-seed}_{\text{Token}}[e]$ for each epoch e. It should reflect whether $\text{seed}_{\text{Token}}$ from the challenge ciphertext of the current epoch is known. Initially $\text{know-seed}_{\text{Token}}[e]$ is false for all epochs e. Whenever the challenge ciphertext is output to the adversary (when creating the challenge and when O_{UpdC} is called) set $\text{know-seed}_{\text{Token}}[e] \leftarrow$ true. When O_{Corr} is called in epoch $e \geq \tilde{e}$ to reveal the tokens, do one of the following:
  - If $\text{know-seed}_{\text{Token}}[e-1]$ = true then set $\text{know-seed}_{\text{Token}}[e] \leftarrow$ true. *Intuitively, in order to learn the current* $\text{seed}_{\text{Token}}$ *from the token, the* $\text{seed}_{\text{Token}}$ *from the previous epoch is needed to remove the PRG-blinding on the token.*
  - On the other hand if $\text{know-seed}_{\text{Token}}[e-1]$ = false, instead of returning the correct token, O_{Corr} returns a random string of equal length. *The reason why we do this becomes apparent in the next game hop.*

  If $\mathcal{A}$ is not allowed to query O_{Corr} with inp = token (i.e. the bound for PUE-One), then trivially $\Pr[G_1 \Rightarrow 1] = \Pr[G_2 \Rightarrow 1]$. Otherwise we claim $\Pr[G_1 \Rightarrow 1] \leq \Pr[G_2 \Rightarrow 1] + \mathsf{e}_{\max}\text{Adv}^{\text{prg}}_{G}(\lambda,\mathsf{y})$. To see this, when the game in any epoch would use $G(\text{seed}_{\text{Token}})$ in UE.TG for the challenge ciphertext while $\text{know-seed}_{\text{Token}}$ is false for the previous epoch, instead we use the output of a PRG challenger. Since

know-seed$_{\mathsf{Token}}[e-1]$ cannot be changed in epoch e anymore (after all, seed$_{\mathsf{Token}}$ can only be revealed through $O_{\mathsf{Upd}\mathcal{C}}$ for the current epoch), we know at the start of epoch e whether to use the PRG challenger output or the real value $G(\mathsf{seed}_{\mathsf{Token}})$.

- G_3: *Intuition: In this game we replace the ciphertext headers with random. If the epoch's key is not corrupted, we can use AE security. If it is known, we show know-s is false and thus G_2 already randomized the token including the header.* In any epoch $e \geq \tilde{e}$, when UE.TG is called to update the challenge ciphertext, replace the call to AE.Enc(m) with taking a random string of the same length as the real output. Store m and the result in a message-ciphertext table T, and when AE.Dec (in UE.TG) would be called on one of the ciphertexts in T, treat the corresponding m as the decryption. Note that this simulation works as we are in the CPA security game and thus AE.Dec is only called by the experiment on ciphertext headers created by the experiment. For detIND-ENC-CPA and detIND-ENC$-CPA we do the same for the AE.Enc call which produces the header in UE.Enc during challenge ciphertext creation in "Create $\tilde{C}$".
  We claim $\Pr[G_2 \Rightarrow 1] \leq \Pr[G_3 \Rightarrow 1] + \mathsf{e}_{\mathsf{max}}\mathsf{Adv}^{\mathsf{dic}}_{\mathsf{AE}}(\lambda)$. To see this, for any epoch $e \geq \tilde{e}$ construct an adversary against the DIC challenger which uses the first oracle ($\mathsf{Enc}_k(\cdot)$) to replace any regular AE.Enc call from G_2 and use the second oracle ($\mathcal{E}$) replace the special AE.Enc calls listed above (i.e. when UE.TG is called to update the challenge ciphertext, and potentially during challenge ciphertext creation). The O_{Corr} query from G_2 with inp = k uses O_{Corr} by the DIC challenger.
  Now we need to consider the following cases.
  - If the key was not corrupted in epoch e, then the DIC experiment with $b = 1$ exactly corresponds to G_2 (all AE.Enc calls use the same key) and with $b = 0$ corresponds to G_3 (regular AE.Enc calls happen normally and for the challenge ciphertext header and token instead random bitstrings are used). The adversary's chance to detect the change for this epoch thus is $\mathsf{Adv}^{\mathsf{dic}}_{\mathsf{AE}}(\lambda)$.
  - If the key was corrupted in epoch e, we claim that the adversary's chance to detect the change for this epoch is 0. Since the trivial win conditions prevent $\mathcal{A}$ from corrupting the challenge ciphertext for this epoch, the change to it is invisible to $\mathcal{A}$. So only the token could reveal the change. However we claim that know-seed$_{\mathsf{Token}}[e-1]$ = false and the current token was thus randomized by G_2.
    Observe that if there is a chain of consecutive epochs where know-seed$_{\mathsf{Token}}$ is true, then in the first of these epochs the challenge ciphertext must have been revealed (from the challenge creation or $O_{\mathsf{Upd}\mathcal{C}}$). For any of the other epochs, either the challenge ciphertext was revealed or the token was revealed, making the ciphertext inferrable. Therefore the trivial win conditions prevent $\mathcal{A}$ from making queries s.t. know-seed$_{\mathsf{Token}}[e-1]$ = true.

  This proves $\Pr[G_2 \Rightarrow 1] \leq \Pr[G_3 \Rightarrow 1] + \mathsf{e}_{\mathsf{max}}\mathsf{Adv}^{\mathsf{dic}}_{\mathsf{AE}}(\lambda)$. From Lemma 6.8, we have $\Pr[G_2 \Rightarrow 1] \leq \Pr[G_3 \Rightarrow 1] + 2\mathsf{e}_{\mathsf{max}}\mathsf{Adv}^{\mathsf{priv}}_{\mathsf{AE}}(\lambda)$.
- G_4: Modify the creation of the challenge ciphertext in "Create $\tilde{C}$".
  - For IND-ENC$: Replace the SE.Enc call with a random bitstring of the same length. Since the corresponding key k_{Data} does not appear in the system anymore (G_3 randomized all the ciphertext headers), we can embed the PRIV adversary for SE and find
    $$\Pr[G_2 \Rightarrow 1] \leq \Pr[G_3 \Rightarrow 1] + \mathsf{Adv}^{\mathsf{priv}}_{\mathsf{SE}}(\lambda)$$
  - For IND-UPD and IND-UPD$: Replace the $G(\mathsf{seed})$ call during UE.Upd call with a random bitstring of the same length. The seed otherwise appears:
    (1) In the update token used for the UE.Upd call, which cannot be corrupted as per trivial win condition.
    (2) Inside the set S, which is only stored in AE ciphertexts which were replaced with random bitstrings in G_3.

    Here $\Pr[G_3 \Rightarrow 1] \leq \Pr[G_4 \Rightarrow 1] + \mathsf{Adv}^{\mathsf{prg}}_{G}(\lambda, \gamma)$.

  For IND-ENC$, we can start with the original game for $b = 0$ and apply the same steps to arrive at G_4. The proof for IND-ENC$ ends here, the final advantage is twice the difference between G_0 and G_4. We continue for the proofs of the other security notions.
  In G_4, "Create $\tilde{C}$" returns a random ciphertext similar to the original game for $b = 0$ of IND-ENC$, IND-UPD$. The only thing left to consider are the corrupted update tokens. However they also do not help a distinguisher:
  - For PUE-List, the token is of the form $\Delta = ((\mathsf{seed}_{\mathsf{Token}}', x'), s)$, where $\mathsf{seed}_{\mathsf{Token}}'$ and s were randomly drawn in TG independent of b. x' is AE encrypted and was replaced with random in G_3.
  - For PUE-One, tokens are not allowed to be corrupted.

  Thus G_4 cannot be distinguished from the original game with $b = 0$.

□

7 Performance

In Table 1 (extended in Table 6), we benchmarked our C implementation [3] of PUE-One and PUE-List (for 50 updates) on an Intel Core i7-11700F (2.50GHz) using WSL with Ubuntu and OpenSSL for AES-based implementations. We compared it to the public implementations of the Nested scheme and KH-PRF "UAE 60" [8] and Recrypt [15] (ReCrypt). SHINE [10] has no implementation, we estimate performance similar to ReCrypt due to a similar number of exponentiations. We encrypted 32 KB messages.

All PUE-One functions are at most 1.6x slower than AES-GCM (12.88 cycles per byte (cpb)), unlike the at least 5x slower encryption/decryption in prior UE schemes. PUE-List has 1500x faster

Scheme	Max Upd.	Security Level	Encrypt	Decrypt (1 - 50 u)	Token	Update	Overhead
SHINE [10]	∞	CCA	$\approx$ 15000	$\approx$ 13000	$\approx 10^7$	$\approx$ 12000	3%
PUE-List (this work)	50	CCA	19.63	19.07 391.56	30543	7.78	3%
ReCrypt [15]	∞	CPA	15431	13745	$1.1 \cdot 10^7$	12945	3%
KH-PRF [8]	∞	CPA	60.06	33.72	314836	19.80	36%
Nested [8]	50	CPA, no token	20.43	20.35 405.18	36342	10.27	7%
PUE-One (this work)	∞	CCA, no token	20.04	16.94	21268	14.53	0.3%

Table 6: Performance, security and ciphertext expansion of UE schemes.
We grouped the schemes by security level, with the high security schemes in the first box. "no token " refers to restricting the adversary from corrupting tokens. The "Token" column lists the total time for generating a new key and the corresponding update token. Measurements are given in cycles per byte, except for "Token" which is given in cycles (total). Ciphertext expansion is considered for 32KB messages. From - to values refer to the size after $u = 1$ to 50 updates.

updates, 600x faster encryption and 600x-30x faster decryptions than SHINE. PUE-One has 24x faster decryption than Nested at 50 updates, similar encryption and 46% slower updates. PUE-One has the lowest ciphertext overhead at 0.3%.

HES, KSS and 2ENC have no implementation available. HES and KSS update in constant time, with encryption/decryption performance mainly limited by AES-GCM (12.88 cpb) or slower schemes, making them up to 1.6x faster than PUE-One. 2ENC encryptions/decryptions/updates require two AES-CTR passes (11.43 cpb), giving 22.86 cpb, i.e. 1.1x/1.3x/1.6x slower than PUE-One.

Benchmark comparison. PUE-List **performance.** SHINE [10] is the only comparable scheme security-wise but is outperformed by PUE-List — 600x faster encryption/decryption (after 1 update) and 30x faster decryption (after 50 updates). SHINE becomes faster only after 1500 updates. PUE-List match the Nested scheme[8] in encryption/decryption speed and have 25% faster updates.

PUE-One **performance.** PUE-One has similar security to Nested [8], KH-PRF, and ReCrypt [15]. It matches Nested in encryption speed, $\approx$3x faster than KH-PRF, and maintains constant decryption speed for $u \geq 1$, making it 24x faster than Nested at 50 updates. PUE-One is the only symmetric scheme faster than all asymmetric ones for any update count but has 41% slower updates than Nested.

Prior benchmarks. Our results roughly match the ReCrypt [15] benchmarks by Everspaugh et al. [15] as well as the Nested scheme [8] and KH-PRF [8] benchmarks by Boneh et al. [8]. The only significant difference is that KH-PRF compares much more favorably to Nested in our benchmark than in the one by Boneh et al., having faster decryptions already after 4 updates rather than 50 as they suggested. We suspect these differences occur from running the benchmarks on different machines with different levels of hardware support for cryptographic operations. This discrepancy has little impact on our overall results, since our scheme PUE-One outperforms both Nested and KH-PRF (even with its improved performance) in terms of decryption time.

Acknowledgements

We thank the anonymous reviewers for their helpful comments and suggestions. Elena Andreeva and Andreas Weninger are supported in part by the Austrian Science Fund (FWF) SFB project SPyCoDe 10.55776/F85.

References

[1] 2025. AWS Encryption. https://docs.aws.amazon.com/AmazonS3/latest/userguide/UsingClientSideEncryption.html. Accessed 21-March-2025.

[2] Navid Alamati, Hart Montgomery, and Sikhar Patranabis. 2019. Symmetric Primitives with Structured Secrets. In *Advances in Cryptology – CRYPTO 2019, Part I (Lecture Notes in Computer Science, Vol. 11692)*, Alexandra Boldyreva and Daniele Micciancio (Eds.). Springer, Cham, 650–679. doi:10.1007/978-3-030-26948-7_23

[3] Andreas Weninger. 2025. PUE-Implementation: Source code repository. https://github.com/EthylPropan/PUE-Implementation.

[4] Elena Andreeva, Benoît Cogliati, Virginie Lallemand, Marine Minier, Antoon Purnal, and Arnab Roy. 2024. Masked Iterate-Fork-Iterate: A New Design Paradigm for Tweakable Expanding Pseudorandom Function. In *Applied Cryptography and Network Security – ACNS 2024 (Lecture Notes in Computer Science, Vol. 14584)*. Springer, 433–459. doi:10.1007/978-3-031-54773-7_17

[5] Elena Andreeva, Virginie Lallemand, Antoon Purnal, Reza Reyhanitabar, Arnab Roy, and Damian Vizár. 2019. Forkcipher: A New Primitive for Authenticated Encryption of Very Short Messages. In *Advances in Cryptology – ASIACRYPT 2019, Part II (Lecture Notes in Computer Science, Vol. 11922)*, Steven D. Galbraith and Shiho Moriai (Eds.). Springer, Cham, 153–182. doi:10.1007/978-3-030-34621-8_6

[6] Elena Andreeva, Reza Reyhanitabar, Kerem Varici, and Damian Vizár. 2018. Forking a Blockcipher for Authenticated Encryption of Very Short Messages. IACR Cryptology ePrint Archive, Report 2018/916. https://eprint.iacr.org/2018/916

[7] Elaine Barker. 2020. *Recommendation for Key Management: Part 1 - General (NIST Special Publication 800-57 Part 1 Revision 5)*. NIST Special Publication SP 800-57 Pt. 1 Rev. 5. National Institute of Standards and Technology. doi:10.6028/NIST.SP.800-57pt1r5

[8] Dan Boneh, Saba Eskandarian, Sam Kim, and Maurice Shih. 2020. Improving Speed and Security in Updatable Encryption Schemes. In *Advances in Cryptology – ASIACRYPT 2020, Part III (Lecture Notes in Computer Science, Vol. 12493)*, Shiho Moriai and Huaxiong Wang (Eds.). Springer, Cham, 559–589. doi:10.1007/978-3-030-64840-4_19

[9] Dan Boneh, Kevin Lewi, Hart William Montgomery, and Ananth Raghunathan. 2013. Key Homomorphic PRFs and Their Applications. In *Advances in Cryptology – CRYPTO 2013, Part I (Lecture Notes in Computer Science, Vol. 8042)*, Ran Canetti and Juan A. Garay (Eds.). Springer, Berlin, Heidelberg, 410–428. doi:10.1007/978-3-642-40041-4_23

[10] Colin Boyd, Gareth T. Davies, Kristian Gjøsteen, and Yao Jiang. 2020. Fast and Secure Updatable Encryption. In *Advances in Cryptology – CRYPTO 2020, Part I (Lecture Notes in Computer Science, Vol. 12170)*, Daniele Micciancio and Thomas Ristenpart (Eds.). Springer, Cham, 464–493. doi:10.1007/978-3-030-56784-2_16

[11] Huanhuan Chen, Yao Jiang Galteland, and Kaitai Liang. 2023. CCA-1 Secure Updatable Encryption with Adaptive Security. In *Advances in Cryptology – ASIACRYPT 2023, Part V (Lecture Notes in Computer Science, Vol. 14442)*, Jian Guo and Ron Steinfeld (Eds.). Springer, Singapore, 374–406. doi:10.1007/978-981-99-8733-7_12

[12] Long Chen, Yanan Li, and Qiang Tang. 2020. CCA Updatable Encryption Against Malicious Re-encryption Attacks. In *Advances in Cryptology – ASIACRYPT 2020, Part III (Lecture Notes in Computer Science, Vol. 12493)*, Shiho Moriai and Huaxiong Wang (Eds.). Springer, Cham, 590–620. doi:10.1007/978-3-030-64840-4_20

[13] PCI Security Standards Council. 2018. Payment card industry data security standard.

[14] Morris J. Dworkin. 2007. *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*. NIST Special Publication 800-38D. National Institute of Standards and Technology, Gaithersburg, MD. doi:10.6028/NIST.SP.800-38D

[15] Adam Everspaugh, Kenneth G. Paterson, Thomas Ristenpart, and Samuel Scott. 2017. Key Rotation for Authenticated Encryption. In *Advances in Cryptology – CRYPTO 2017, Part III (Lecture Notes in Computer Science, Vol. 10403)*, Jonathan Katz and Hovav Shacham (Eds.). Springer, Cham, 98–129. doi:10.1007/978-3-319-63697-9_4

[16] Federal Reserve Board. 2023. Records Retention, Reserve Bank Oversight. https://www.federalreserve.gov/foia/rr_rboversight.htm Accessed: 2025-08-21.

[17] Centers for Medicare and Medicaid Services. 2025. CMS Key Management Handbook. https://security.cms.gov/learn/cms-key-management-handbook#key-rotation Accessed 25-August-2025.

[18] Google. 2025. Google Encryption. https://cloud.google.com/storage/docs/encryption. Accessed 21-March-2025.

[19] Google. 2025. Google Envelope Encryption. https://cloud.google.com/kms/docs/envelope-encryption. Accessed 25-August-2025.

[20] Yao Jiang. 2020. The Direction of Updatable Encryption Does Not Matter Much. In *Advances in Cryptology – ASIACRYPT 2020, Part III (Lecture Notes in Computer Science, Vol. 12493)*, Shiho Moriai and Huaxiong Wang (Eds.). Springer, Cham, 529–558. doi:10.1007/978-3-030-64840-4_18

[21] Michael Klooß, Anja Lehmann, and Andy Rupp. 2019. (R)CCA Secure Updatable Encryption with Integrity Protection. In *Advances in Cryptology – EUROCRYPT 2019, Part I (Lecture Notes in Computer Science, Vol. 11476)*, Yuval Ishai and Vincent Rijmen (Eds.). Springer, Cham, 68–99. doi:10.1007/978-3-030-17653-2_3

[22] Anja Lehmann and Björn Tackmann. 2018. Updatable Encryption with Post-Compromise Security. In *Advances in Cryptology – EUROCRYPT 2018, Part III (Lecture Notes in Computer Science, Vol. 10822)*, Jesper Buus Nielsen and Vincent Rijmen (Eds.). Springer, Cham, 685–716. doi:10.1007/978-3-319-78372-7_22

[23] Peihan Miao, Sikhar Patranabis, and Gaven J. Watson. 2023. Unidirectional Updatable Encryption and Proxy Re-encryption from DDH. In *PKC 2023: 26th International Conference on Theory and Practice of Public Key Cryptography, Part II (Lecture Notes in Computer Science, Vol. 13941)*, Alexandra Boldyreva and Vladimir Kolesnikov (Eds.). Springer, Cham, 368–398. doi:10.1007/978-3-031-31371-4_13

[24] Ryo Nishimaki. 2022. The Direction of Updatable Encryption Does Matter. In *PKC 2022: 25th International Conference on Theory and Practice of Public Key Cryptography, Part II (Lecture Notes in Computer Science, Vol. 13178)*, Goichiro Hanaoka, Junji Shikata, and Yohei Watanabe (Eds.). Springer, Cham, 194–224. doi:10.1007/978-3-030-97131-1_7

[25] Atul Purnal, Elena Andreeva, Arnab Roy, and Damian Vizár. 2019. What the Fork: Implementation Aspects of a Forkcipher. In *NIST Lightweight Cryptography Workshop*.